

60分で学ぶ数理最適化

大阪大学 大学院情報科学研究科
梅谷 俊治

2020年6月16日
(6月23日修正)

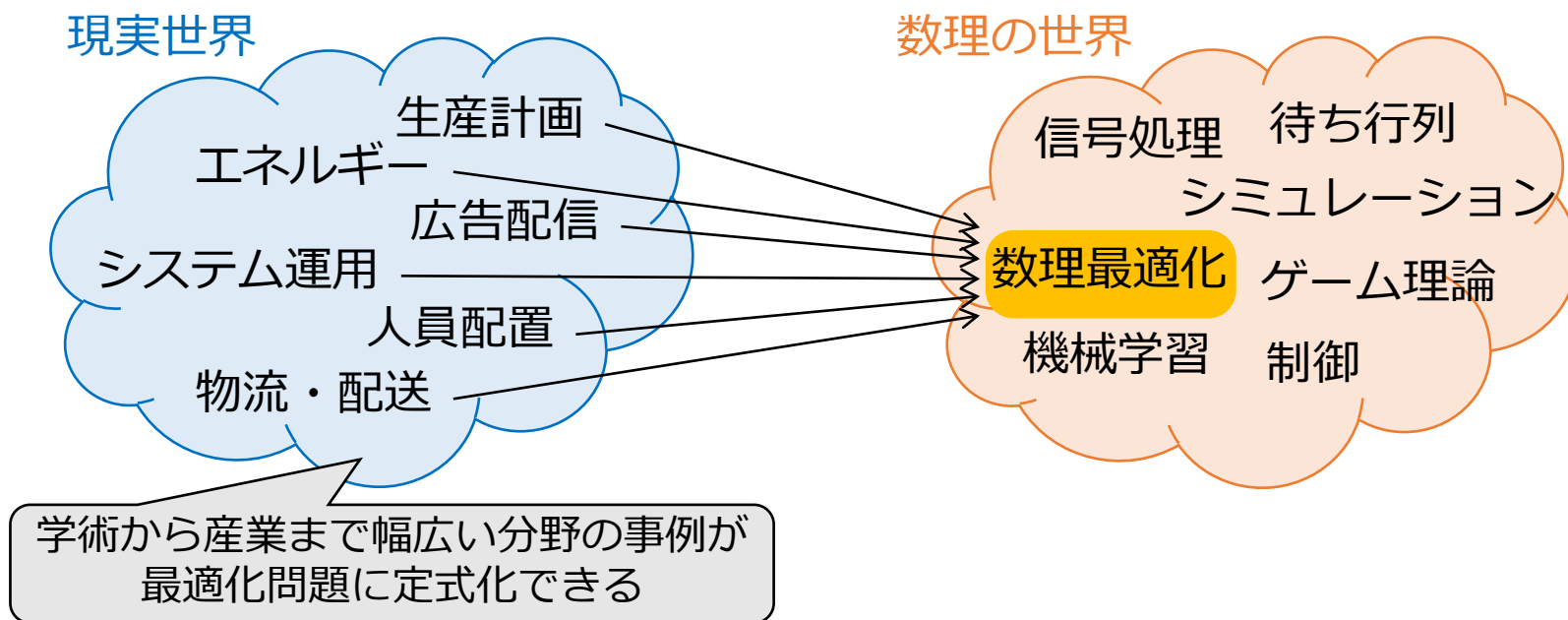
はじめに

- 産業や学術など幅広い分野における多くの重要な現実問題が数理最適化として定式化できる.
- 近年では, 数理最適化アルゴリズムの進歩が計算機の性能向上と相まって, 以前では計算困難であった大規模・複雑な現実問題が扱えるようになった.
- 数理最適化の基本的な枠組みを学習すれば, 将来, 多様な応用分野において問題を解決するための有用な手段を身につけることができる.
- 本スライドでは, 数理最適化の枠組みを概観することで, 数理最適化を本格的に学習するきっかけを与えることを目的とする.

注意: このスライドでは, 双対問題をはじめとする多くの重要な概念の説明を省略しています. もし, このスライドを読み終えて数理最適化を深く理解できたと感じたなら, それはたぶん気のせいです.

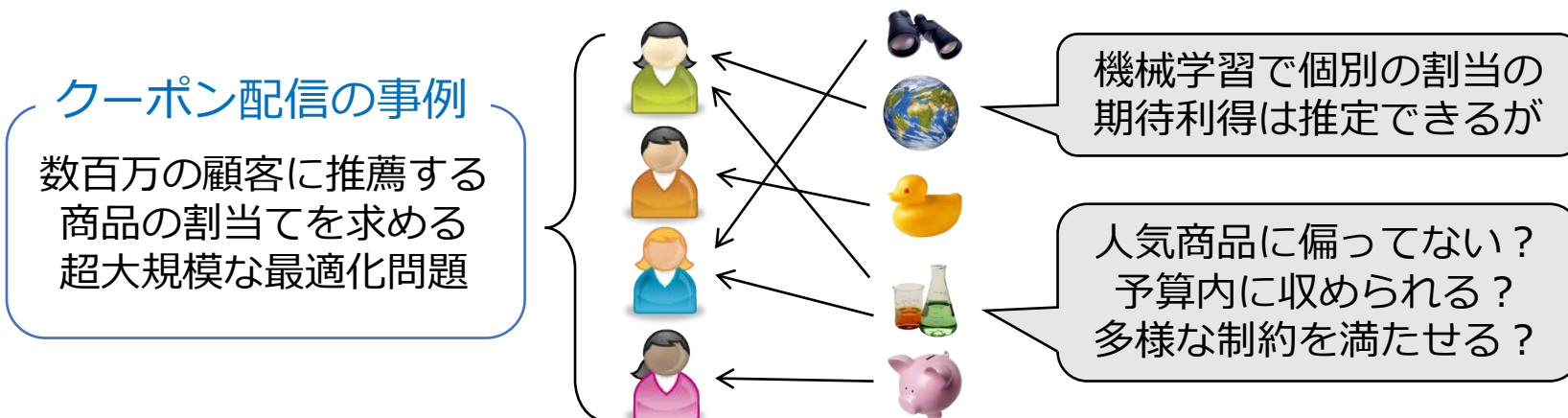
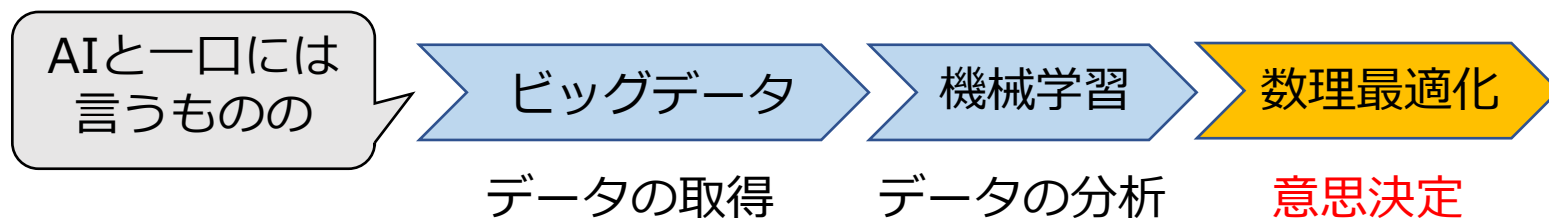
数理最適化による問題解決

- ビッグデータが取得・処理できた！ビッグデータが分析できた！ところで、どうやって意思決定・問題解決を実現すれば良い？
- 数理最適化は意思決定・問題解決のための有効な手段.
- **最適化問題**：制約条件を満たす解の中で目的関数の値を最小(最大)にする解を求める問題.



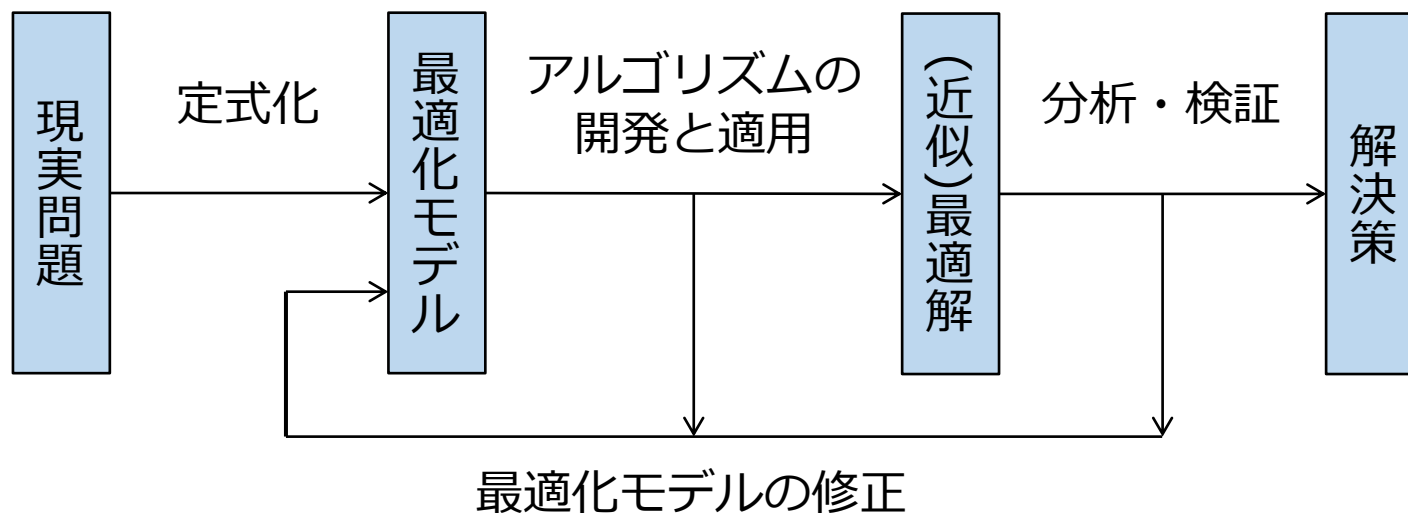
数理最適化による問題解決

- ビッグデータが取得・処理できた！ビッグデータが分析できた！ところで、どうやって意思決定・問題解決を実現すれば良い？
- 数理最適化は意思決定・問題解決のための有効な手段.
- **最適化問題**：制約条件を満たす解の中で目的関数の値を最小(最大)にする解を求める問題.



数理最適化による問題解決

- ビッグデータが取得・処理できた！ビッグデータが分析できた！ところで、どうやって意思決定・問題解決を実現すれば良い？
- 数理最適化は意思決定・問題解決のための有効な手段.
- **最適化問題**： **制約条件**を満たす解の中で**目的関数**の値を最小(最大)にする解を求める問題.



数理最適化の応用事例*

- 石油・化学産業：配送，資源割当て，配合，販売など.
- 製造業：資源割当て(生産能力，原料，人員など)など.
- 運輸業(トラック，航空機，船舶，鉄道，バスなど)：配送，スケジューリング，拠点配置など.
- 経理財務：ポートフォリオ選択，税収構成，与信管理など.
- 農業：作付，飼料配合，配送，販売価格の決定など.
- 医療：看護師スケジューリング，医療・検査機器の調整など.
- 鉱業：資源割当て(機械，人員など)，鉱石配合，操業計画など.
- 人員計画：配置転換，人員管理など.
- 食品：原料配合，配送，資源割当てなど.
- エネルギー(電力，ガスなど)：発電スケジューリング，配送，販売価格の決定など.
- 製紙業：配合，リサイクルなど.
- 広告業(テレビ，新聞など)：予算配分，スケジューリングなど.

最適化問題に定式化する

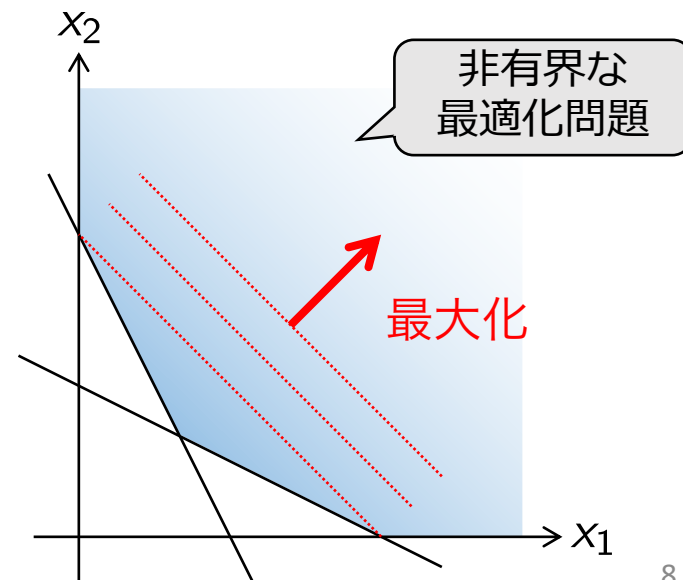
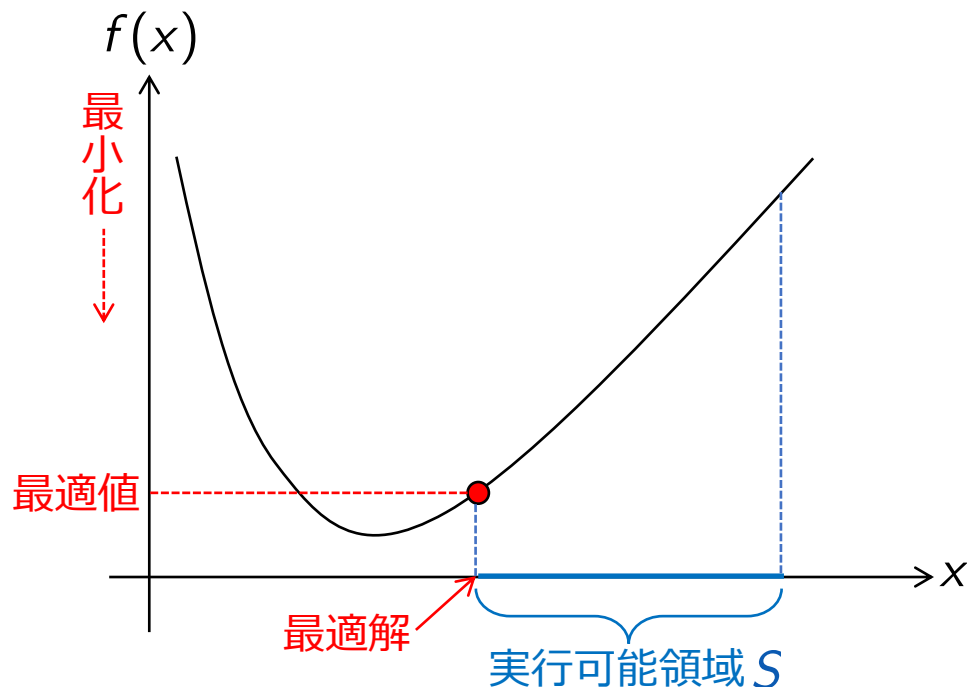
- ある飲料メーカーでは、トマト、にんじん、ほうれん草を原料とする野菜ジュースを製造している。
- 野菜ジュースに含まれる食物繊維、ビタミンC、鉄分、βカロチンの必要量を満たしつつ、製造に要する原料費を最小に抑えるための各野菜の購入量は？
- トマト、にんじん、ほうれん草の購入量(kg)を x_1, x_2, x_3 とする。

	食物繊維	ビタミンC	鉄分	βカロチン	価格(円/kg)
トマト	10	15	2	5	400
にんじん	25	5	2	80	250
ほうれん草	30	35	20	40	1000
必要量(単位/2L)	50	60	10	40	

minimize $400x_1 + 250x_2 + 1000x_3$ → 原料費を最小化
 subject to $10x_1 + 25x_2 + 30x_3 \geq 50$, → 食物繊維は50単位以上
 $15x_1 + 5x_2 + 35x_3 \geq 60$, → ビタミンCは60単位以上
 $2x_1 + 2x_2 + 20x_3 \geq 10$, → 鉄分は10単位以上
 $5x_1 + 80x_2 + 40x_3 \geq 40$, → βカロチンは40単位以上
 $x_1, x_2, x_3 \geq 0$. → 各野菜の購入量は非負

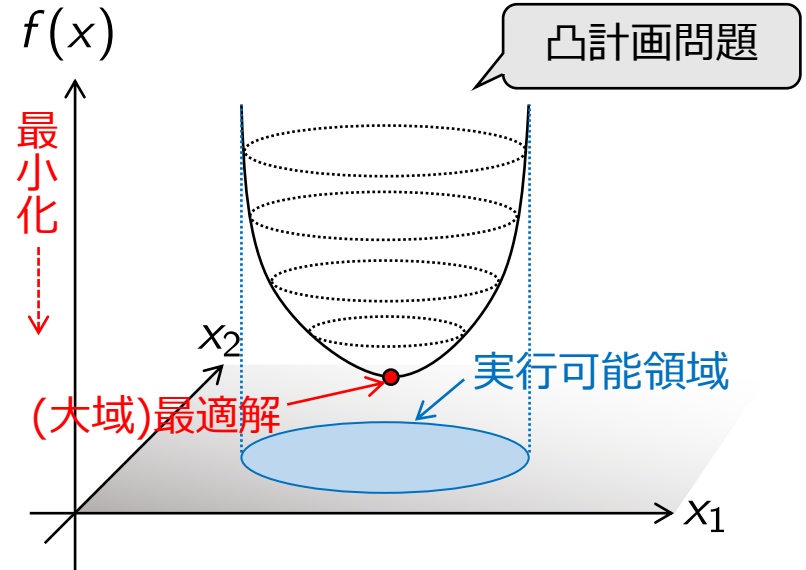
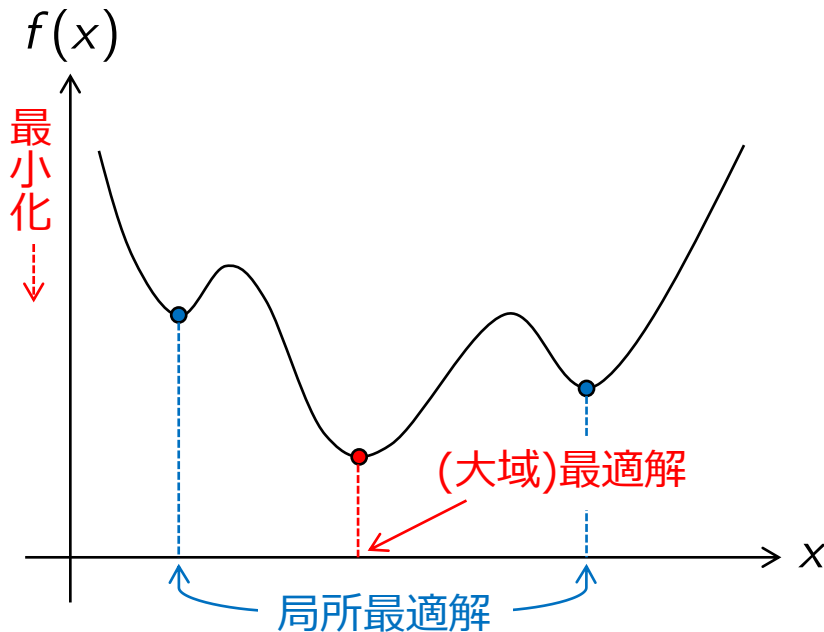
最適化問題

- 実行可能解：制約条件を満たす解
- 実行可能領域：実行可能解全体の集合
- 最適解：目的関数の値が最小(最大)となる実行可能解
- 最適値：最適解における目的関数の値
- 実行不能：制約条件を満たす解が存在しない
- 非有界：目的関数の値を限りなく改善できるため最適解が存在しない



最適化問題

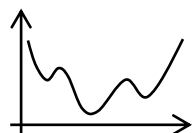
- $\mathbf{x}$ が(大域)最適解：任意の実行可能解 $\mathbf{y}$ に対して $f(\mathbf{x}) \leq f(\mathbf{y})$
- $\mathbf{x}$ が局所最適解： $\mathbf{x}$ の近傍内の任意の実行可能解 $\mathbf{y}$ に対して
- 目的関数が凸関数かつ実行可能領域が凸集合 → 凸計画問題
→ 局所最適解=(大域)最適解
→ 効率的なアルゴリズムが多く開発されている。



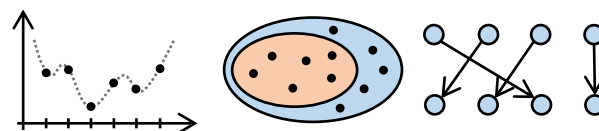
代表的な最適化問題

- 連続最適化問題：変数が実数値のような連続的な値を取る最適化問題
- 離散最適化問題(組合せ最適化問題)：変数が整数値や $\{0,1\}$ の2値のような離散的な値を取る最適化問題や、最適解を含む解の集合が順列やネットワークなど組合せ的な構造を持つ最適化問題。

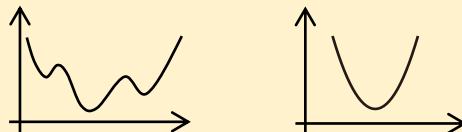
連続最適化問題



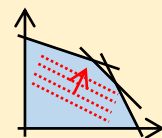
離散最適化問題
(組合せ最適化問題)



非線形計画問題 凸2次計画問題



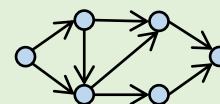
線形計画問題



整数計画問題 混合整数計画問題



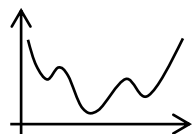
ネットワーク最適化問題



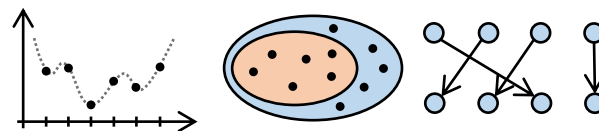
代表的な最適化問題

- **線形計画問題**：目的関数が線形関数で，全ての制約条件が線形の等式もしくは不等式で表された最適化問題
- **2次計画問題**：目的関数が2次関数で，全ての制約条件が線形の等式もしくは不等式で表された最適化問題
- **非線形計画問題**：非線形関数で表された目的関数や制約条件を含む最適化問題

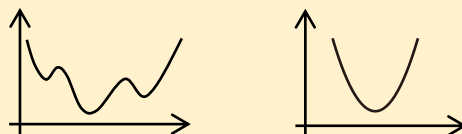
連続最適化問題



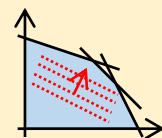
離散最適化問題
(組合せ最適化問題)



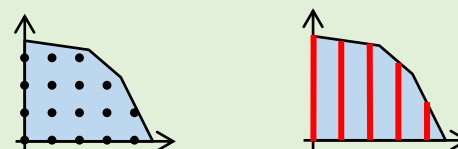
非線形計画問題 凸2次計画問題



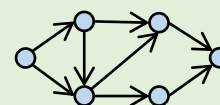
線形計画問題



整数計画問題 混合整数計画問題

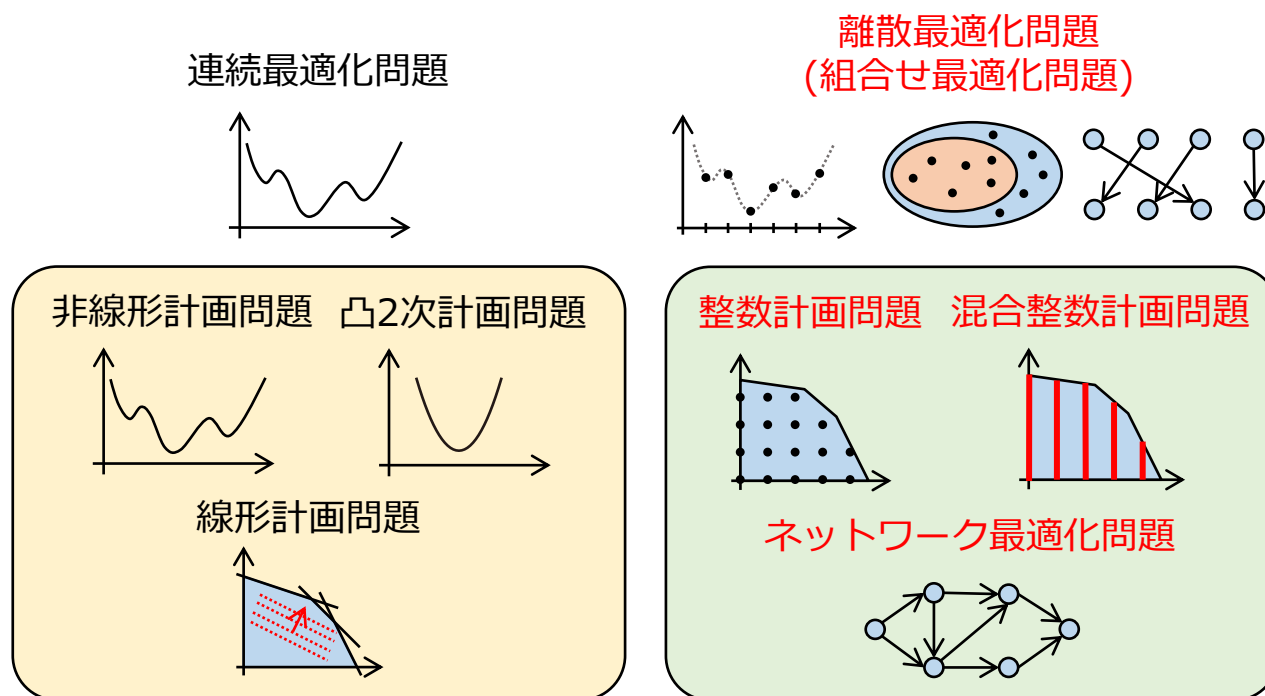


ネットワーク最適化問題



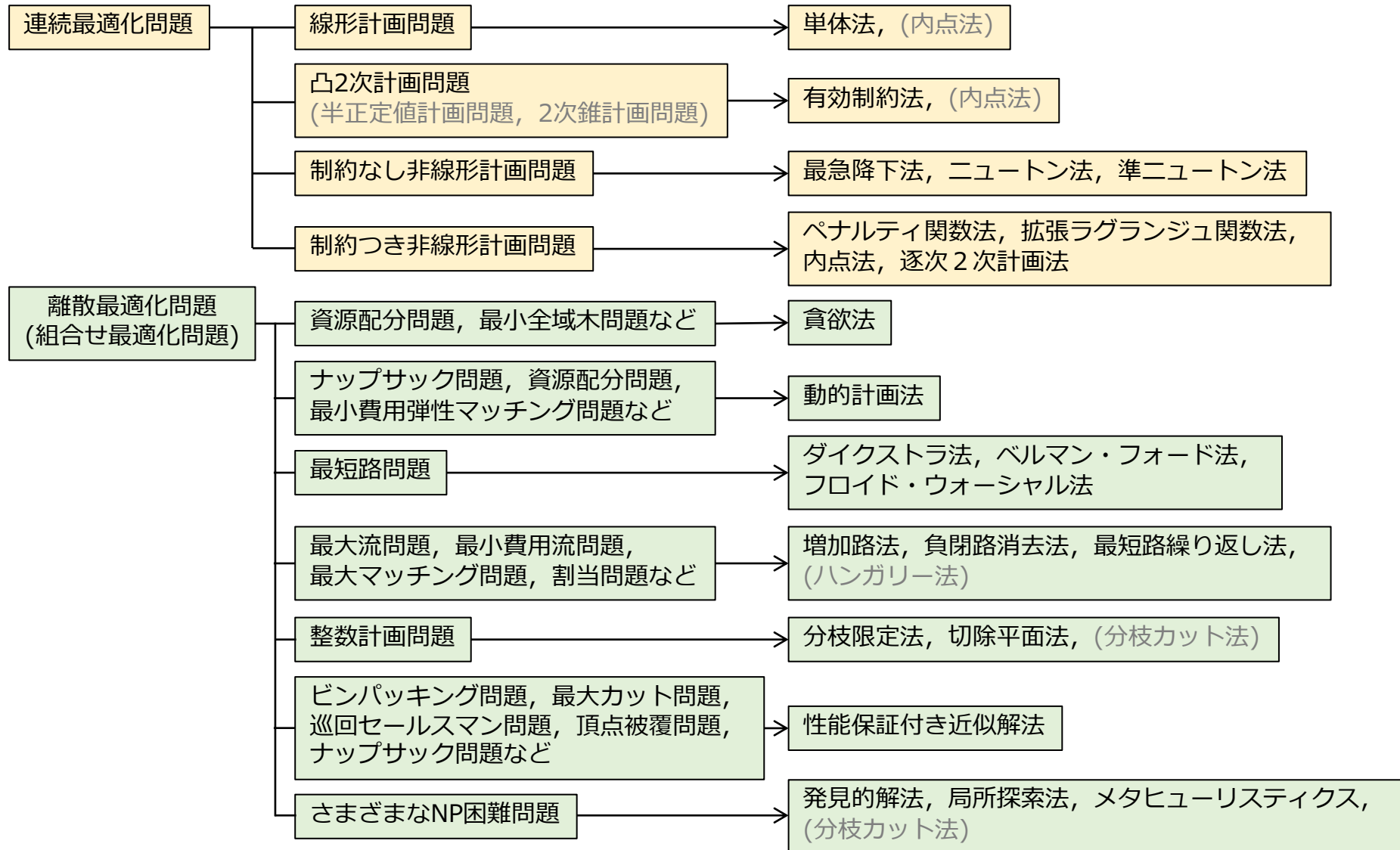
代表的な最適化問題

- 整数計画問題：全ての変数が整数値のみを取る線形計画問題
- 混合整数計画問題*：一部の変数が整数値のみを取る線形計画問題
- 2値整数計画問題：全ての変数が2値のみを取る線形計画問題
- ネットワーク最適化問題：ネットワーク(グラフ)で表される最適化問題



*最近では、混合整数非線形計画問題と区別するために混合整数線形計画問題と呼ぶことも多い。

代表的な最適化問題とアルゴリズム*



*本スライドで紹介する主な最適化問題とアルゴリズムであってチートシート(虎の巻)ではありません。¹³

線形計画問題

- 目的関数が線形関数で、全ての制約条件が線形の等式もしくはは不等式で表された最適化問題.
- さまざまな形の線形計画問題を扱うのは面倒 → (不等式)標準形
- 目的関数は **最大化**, 制約条件は **左辺 ≤ 右辺**, 各変数は**非負**.
- 全ての線形計画問題は標準形に変形できる.

$$\begin{array}{ll}\max. & c_1x_1 + c_2x_2 + \dots + c_nx_n \\ \text{s.t.} & a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \leq b_1, \\ & a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \leq b_2, \\ & \dots \\ & a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \leq b_m, \\ & x_1, x_2, \dots, x_n \geq 0.\end{array}$$

$$\max\{\mathbf{c}^\top \mathbf{x} \mid \mathbf{Ax} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$$

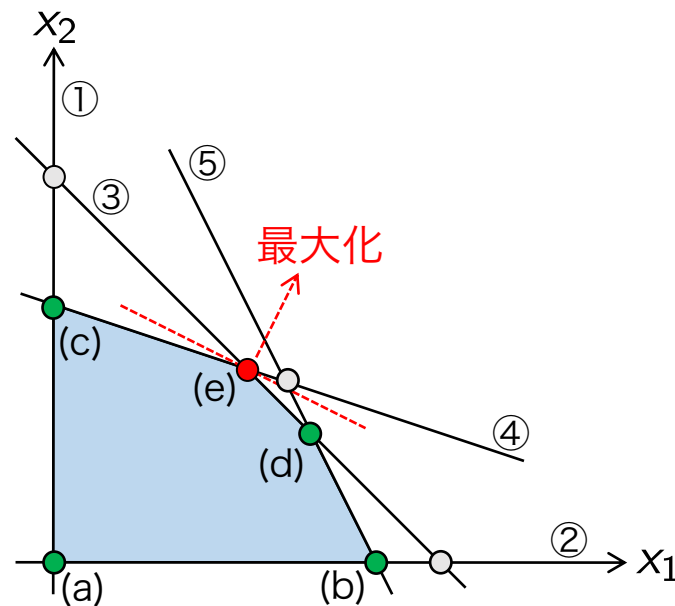
$$\begin{array}{ll}\max. & \sum_{j=1}^n c_j x_j \\ \text{s.t.} & \sum_{j=1}^n a_{ij} x_j \leq b_i, \quad i = 1, \dots, m, \\ & x_j \geq 0, \quad j = 1, \dots, n.\end{array}$$

$$\begin{array}{ll}\max. & \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} & \mathbf{Ax} \leq \mathbf{b}, \\ & \mathbf{x} \geq \mathbf{0}.\end{array}$$

線形計画問題を解く

- 制約条件を m 本, 変数を n 個とする.
- 実行可能領域は $(m + n)$ 枚の超平面に囲まれる → 凸多面体
- n 枚の超平面が交差する頂点のいずれかに最適解がある*.

$$\begin{array}{ll} \text{max.} & x_1 + 2x_2 \\ \text{s.t.} & x_1 \geq 0, \rightarrow \textcircled{1} \\ & x_2 \geq 0, \rightarrow \textcircled{2} \\ & x_1 + x_2 \leq 6, \rightarrow \textcircled{3} \\ & x_1 + 3x_2 \leq 12, \rightarrow \textcircled{4} \\ & 2x_1 + x_2 \leq 10. \rightarrow \textcircled{5} \end{array}$$

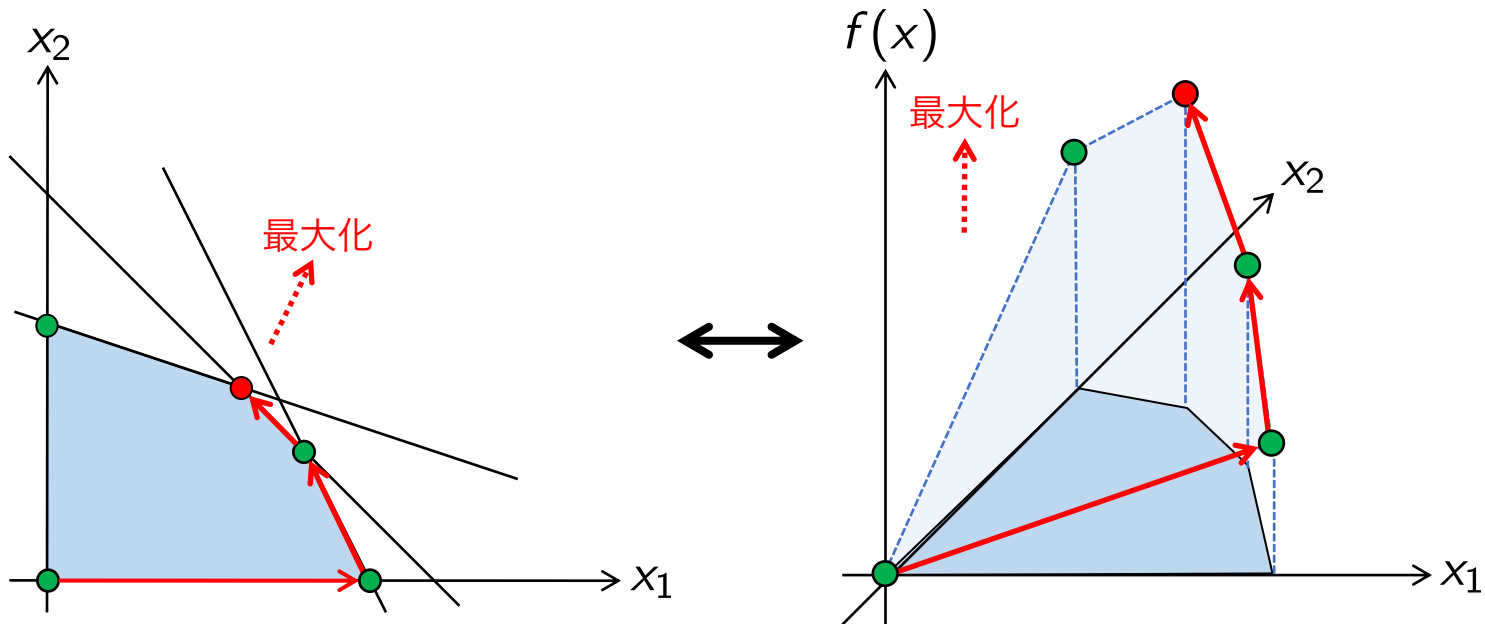


調べる頂点の数が $\binom{m+n}{n} = \frac{(m+n)!}{m!n!}$ 個となり効率的ではない

*実行可能領域が非有界ならば $\max\{x_1 + x_2 \mid x_1 + x_2 \leq 1\}$ のように凸多面体が頂点を持たない場合もあるので注意する.

単体法

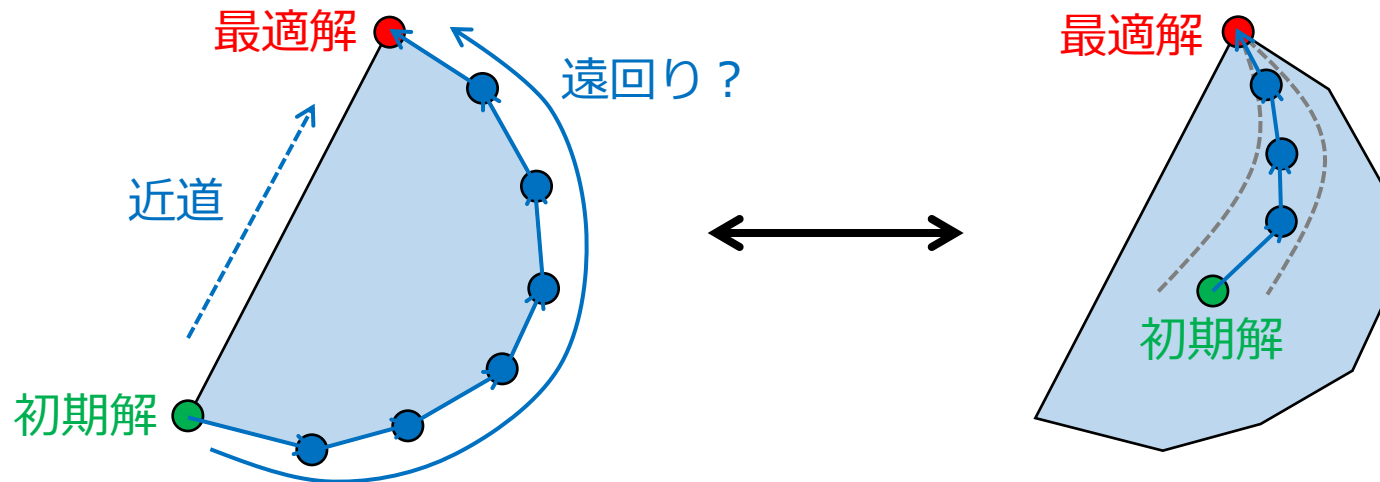
- 凸多面体のある頂点(実行可能解)から出発し, 目的関数の値が改善する隣接頂点への移動を繰り返す.
- 各頂点では n 枚の超平面が交差しているので, n 本の線形式からなる連立1次方程式を解けば対応する実行可能解が得られる*.



*単体法の実装では, 連立1次方程式を一から解き直さなくても済むように工夫されている.

内点法

- 単体法：実行可能領域の境界を通り最適解にたどり着く。
 - ✓ 実用的には高速だが，理論的には最悪で指数時間かかる例も知られている。
- 内点法：実行可能領域の内部を通り最適解にたどり着く。
 - ✓ 実用的に高速で，理論的にも最悪で多項式時間に抑えられる。
- 大規模な問題例を1回だけ解くなら内点法が効率的。
- 入力データを変更して解き直す際に，変更前の最適解を出発点として変更後の最適解を求める再最適化では単体法が効率的。



制約なし非線形計画問題

基本的には、局所最適解を求めるアルゴリズム

- 一般に大域最適解の他に多数の局所最適解が存在する.
- 目的関数 $f(\mathbf{x})$ が凸関数ならば、局所最適解は大域最適解となる
- ヘッセ行列 $\nabla^2 f(\mathbf{x})$ が半正定値*ならば $f(\mathbf{x})$ は凸関数
- 現在の解 $\mathbf{x}$ から $\mathbf{d}$ だけ進んだ先の目的関数は

$$f(\mathbf{x} + \mathbf{d}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{d}$$

勾配 $\nabla f(\mathbf{x})$ とヘッセ行列 $\nabla^2 f(\mathbf{x})$ が重要

$$f(\mathbf{x} + \mathbf{d}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{d} + \frac{1}{2} \mathbf{d}^\top \nabla^2 f(\mathbf{x}) \mathbf{d}$$

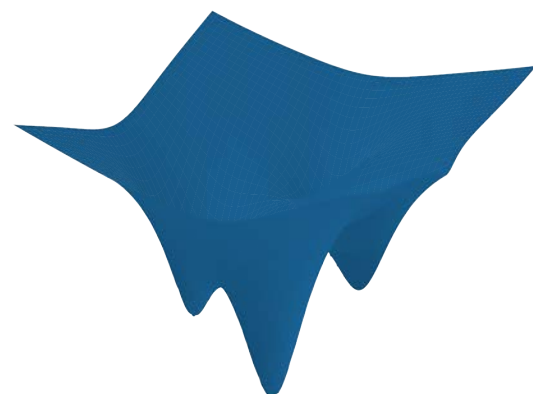
のように、 $\mathbf{d}$ を変数とする線形関数もしくは2次関数に近似できる.

制約なし非線形計画問題

$$\begin{array}{ll} \min. & f(\mathbf{x}) \\ \text{s.t.} & \mathbf{x} \in \mathbb{R}^n. \end{array}$$



凸関数



非凸関数 (多峰性関数)

*任意の $\mathbf{x}$ に対して $\mathbf{x}^\top \nabla^2 f(\mathbf{x}) \mathbf{x} \geq 0$ が成り立つ. 全ての固有値が非負ならば半正定値.

最急降下法

- 反復法：探索方向 $\mathbf{d}$ とステップ幅 $\alpha (> 0)$ を決定し、次の点を求める手続きを繰り返し、局所最適解 $\mathbf{x}^*$ に収束する点列 $\{\mathbf{x}^{(k)}\}$ を生成する*.

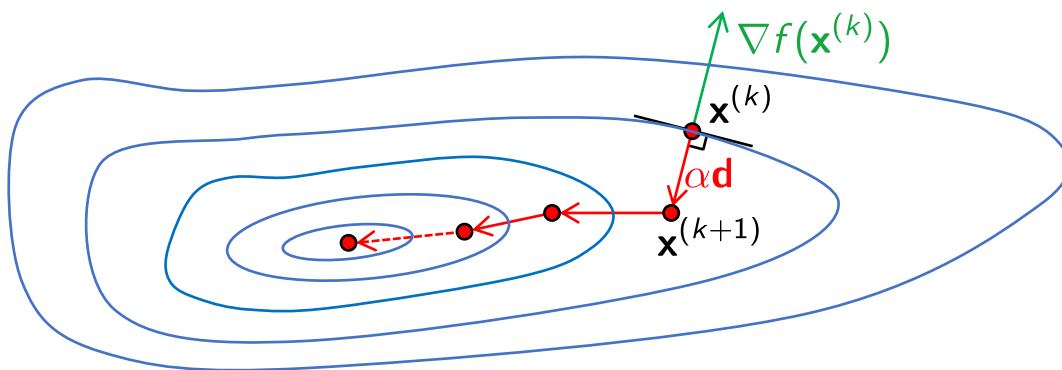
$$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha \mathbf{d}$$

$\nabla f(\mathbf{x}^{(k)})$ と $\mathbf{d}$ が鈍角をなす

- $\nabla f(\mathbf{x}^{(k)})^\top \mathbf{d} < 0$ を満たす探索方向 $\mathbf{d}$ が望ましい.
- 最急降下法では $\mathbf{d} = -\nabla f(\mathbf{x}^{(k)})$ と設定する.
- 直線探索：1変数の最適化問題を近似的に解いてステップ幅 α を求める.

$$\min\{f(\mathbf{x}^{(k)} + \alpha \mathbf{d}) \mid \alpha > 0\}$$

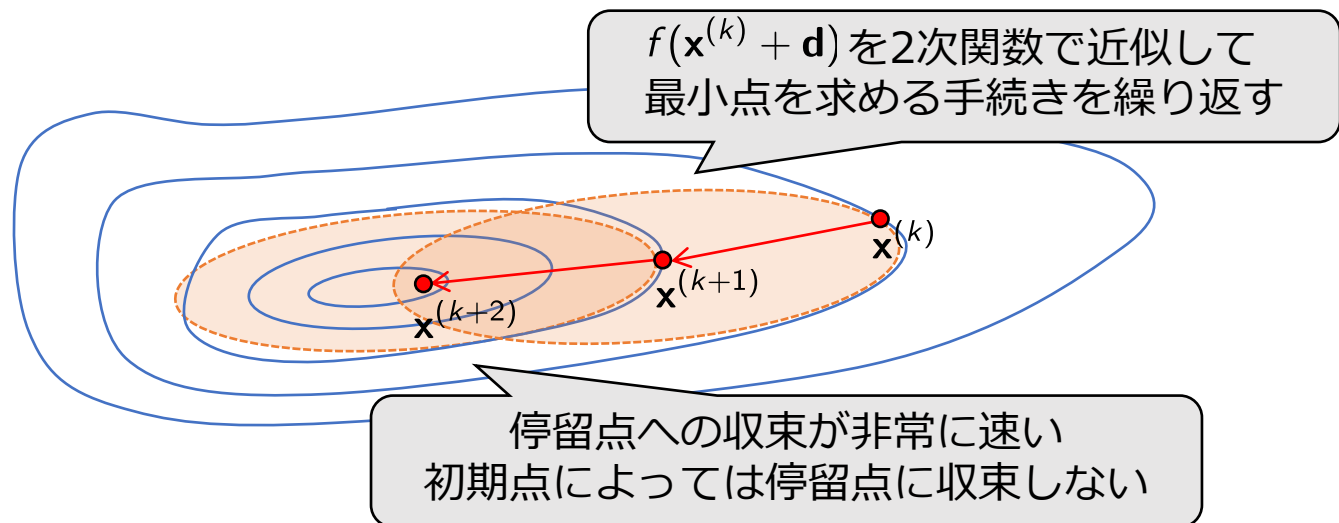
アルミホ条件やウルフ条件を満たす解を2分探索などで求める



正確には、局所最適解ではなく $\nabla f(\mathbf{x}^) = \mathbf{0}$ を満たす停留点 $\mathbf{x}^*$ に収束する.

ニュートン法

- $\nabla f(\mathbf{x}^*) = \mathbf{0}$ となる停留点 $\mathbf{x}^*$ を求めるには?
 $\nabla f(\mathbf{x}^{(k)} + \mathbf{d}) \approx \nabla f(\mathbf{x}^{(k)}) + \nabla^2 f(\mathbf{x}^{(k)})\mathbf{d} = \mathbf{0}$
を解いて探索方向を $\mathbf{d} = -\nabla^2 f(\mathbf{x}^{(k)})^{-1}\nabla f(\mathbf{x}^{(k)})$ とする.
 $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \mathbf{d}$
- ニュートン法の問題点
 - ✓ ヘッセ行列 $\nabla^2 f(\mathbf{x}^{(k)})$ が正定値とは限らず $\nabla f(\mathbf{x}^{(k)})^\top \mathbf{d} < 0$ を満たせない.
 - ✓ ヘッセ行列 $\nabla^2 f(\mathbf{x}^{(k)})$ が正則とは限らず逆行列が計算できない.
 - ✓ ヘッセ行列 $\nabla^2 f(\mathbf{x}^{(k)})$ が効率的に計算できないことがある.



準ニュートン法

- ヘッセ行列 $\nabla^2 f(\mathbf{x}^{(k)})$ の代わりに近似行列 $\mathbf{B}_k$ を生成する.
- 探索方向は $\mathbf{d} = -\mathbf{B}_k^{-1} \nabla f(\mathbf{x}^{(k)})$
- 直線探索によりステップ幅 α を求めて $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha \mathbf{d}$ とする.
- 各反復ではBFGS公式を用いて $\mathbf{B}_k$ から $\mathbf{B}_{k+1}$ を求める.

$$\mathbf{B}_{k+1} \leftarrow \mathbf{B}_k - \frac{\mathbf{B}_k \mathbf{s}^{(k)} (\mathbf{B}_k \mathbf{s}^{(k)})^\top}{(\mathbf{s}^{(k)})^\top \mathbf{B}_k \mathbf{s}^{(k)}} + \frac{\mathbf{y}^{(k)} (\mathbf{y}^{(k)})^\top}{(\mathbf{s}^{(k)})^\top \mathbf{y}^{(k)}}$$

$$\mathbf{s}^{(k)} = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}, \quad \mathbf{y}^{(k)} = \nabla f(\mathbf{x}^{(k+1)}) - \nabla f(\mathbf{x}^{(k)})$$

- ✓ $\mathbf{B}_{k+1}$ は正定値対称行列.
- ✓ $\mathbf{B}_{k+1}$ はセカント条件を満たす.

$$\mathbf{B}_{k+1} (\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}) = \nabla f(\mathbf{x}^{(k+1)}) - \nabla f(\mathbf{x}^{(k)})$$

この条件は $\nabla f(\mathbf{x}^{(k+1)})$ の近似による.

$$\nabla f(\mathbf{x}^{(k+1)}) \approx \nabla f(\mathbf{x}^{(k)}) + \nabla^2 f(\mathbf{x}^{(k)}) (\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)})$$

制約つき非線形計画問題

- 局所最適解 $\mathbf{x}^*$ で $\nabla f(\mathbf{x}^*) = \mathbf{0}$ を満たすとは限らない.
- $\nabla f(\mathbf{x}^{(k)})^\top \mathbf{d} < 0$ を満たす方向 $\mathbf{d}$ に移動すると制約条件を違反する.
- 局所最適解 $\mathbf{x}^*$ にて $g_i(\mathbf{x}^*) = 0$ ならば, $\nabla g_i(\mathbf{x}^*)^\top \mathbf{d} \leq 0$ を満たす方向 $\mathbf{d}$ に移動しないと制約条件を違反する.
- 最適性の1次の必要条件(KKT条件)

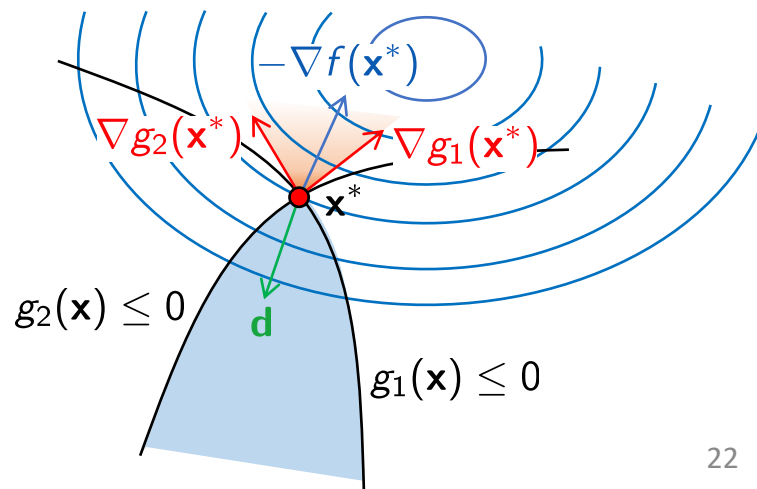
$$\nabla f(\mathbf{x}^*) + \sum_{i \in I(\mathbf{x}^*)} u_i \nabla g_i(\mathbf{x}^*) = \mathbf{0}$$

目的関数を改善しようとする
と制約条件を違反してしまう

- ✓ $g_i(\mathbf{x}^*) = 0$ を満たす制約条件 i の集合 $I(\mathbf{x}^*)$ に着目する.
- ✓ $-\nabla f(\mathbf{x}^*)$ が領域 $G(\mathbf{x}^*) = \{\sum_{i \in I(\mathbf{x}^*)} u_i \nabla g_i(\mathbf{x}^*) \mid u_i \geq 0, i \in I(\mathbf{x}^*)\}$ に含まれる.

制約つき非線形計画問題

$$\begin{array}{ll} \min. & f(\mathbf{x}) \\ \text{s.t.} & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m, \\ & \mathbf{x} \in \mathbb{R}^n. \end{array}$$



有効制約法

- 凸2次計画問題は、機械学習や金融工学など多くの応用例を持つだけでなく、逐次2次計画法などの部分問題にも現れる。
- 現在の解 $\mathbf{x}^{(k)}$ にて $\mathbf{a}_i^\top \mathbf{x}^{(k)} = 0$ を満たす制約条件 i の集合 $I(\mathbf{x}^{(k)})$ からなる部分問題を解き、KKT条件を満たさなければ、現在の解 $\mathbf{x}^{(k)}$ と制約条件 i の集合 $I(\mathbf{x}^{(k)})$ を更新する手続きを繰り返す。

凸2次計画問題

$\mathbf{Q}$ は半正定値

$$\begin{aligned} \min. \quad & \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} \quad & \mathbf{a}_i^\top \mathbf{x} \geq b_i, \quad i = 1, \dots, m, \\ & \mathbf{x} \in \mathbb{R}^n. \end{aligned}$$

部分問題

$$\begin{aligned} \min. \quad & \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} \quad & \mathbf{a}_i^\top \mathbf{x} = b_i, \quad i \in I(\mathbf{x}^*), \\ & \mathbf{x} \in \mathbb{R}^n. \end{aligned}$$

- 部分問題の最適性の十分条件は連立1次方程式になる。

$$\begin{cases} \mathbf{Q} \mathbf{x} + \mathbf{c} + \sum_{i \in I(\mathbf{x}^{(k)})} u_i \mathbf{a}_i = \mathbf{0}, \\ \mathbf{a}_i^\top \mathbf{x} = b_i, \quad i \in I(\mathbf{x}^{(k)}) \end{cases}$$

$u_i < 0$ ならば元の問題(不等式制約)の最適性条件を満たさないので、制約条件 i を集合 $I(\mathbf{x}^{(k)})$ から除外する。

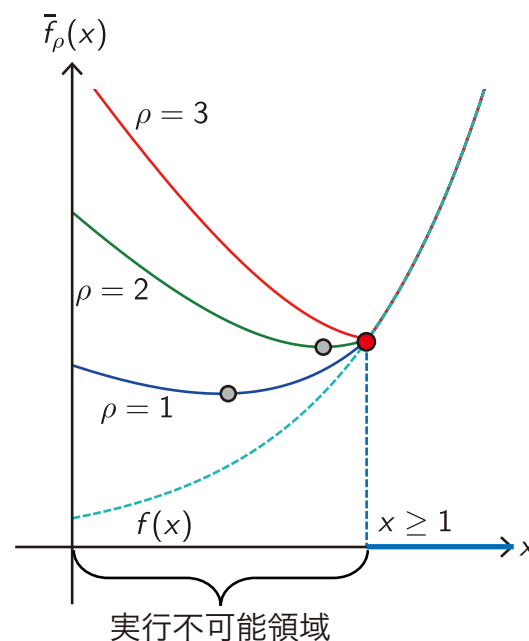
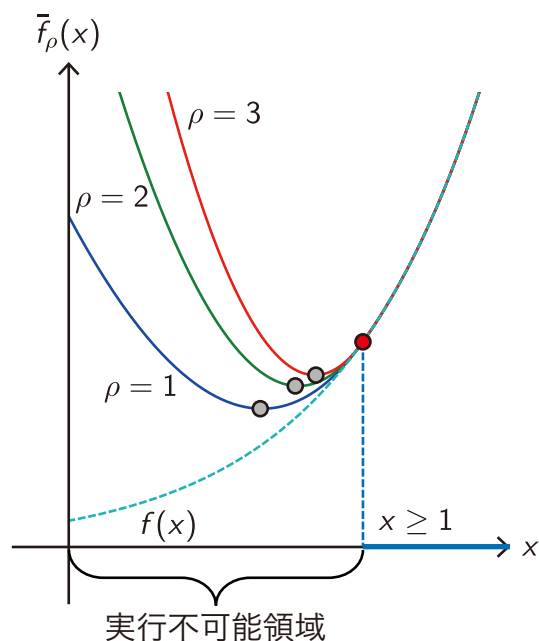
ペナルティ関数法

- 目的関数とペナルティ関数を足し合わせた

$$f_{\rho}(\mathbf{x}) = f(\mathbf{x}) + \rho \sum_{i=1}^m (\max\{0, g_i(\mathbf{x})\})^2 \quad \left[\text{もしくは} + \rho \sum_{i=1}^m |\max\{0, g_i(\mathbf{x})\}| \right]$$

を最小化する制約なし非線形計画問題を解いた後に、ペナルティ重みを更新する手続きを繰り返す。

- ペナルティ関数が2次関数だと局所最適解が得られない、1次関数だと実行可能領域の境界で微分可能ではない。



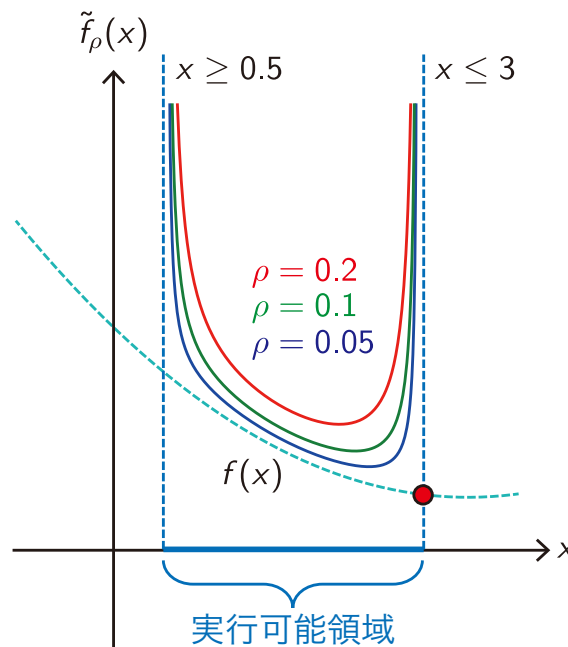
バリア関数法

- 目的関数とバリア関数を足し合わせた

$$f_{\rho}(\mathbf{x}) = \begin{cases} f(\mathbf{x}) - \rho \sum_{i=1}^m \log(-g_i(\mathbf{x})) & \mathbf{g}(\mathbf{x}) < \mathbf{0} \\ +\infty & \mathbf{g}(\mathbf{x}) \not< \mathbf{0} \end{cases}$$

を最小化する制約なし非線形計画問題を解いた後に、ペナルティ重みを更新する手続きを繰り返す。

- 実行可能領域の境界で関数が急峻になり数値的に不安定になる。



拡張ラグランジュ関数法

- ペナルティ関数法の局所最適解に収束しない, 実行可能領域の境界付近で数値的に不安定になる問題を改善する.
- ラグランジュ関数 $L(\mathbf{x}, \mathbf{u}) = f(\mathbf{x}) + \sum_{i=1}^m u_i g_i(\mathbf{x})$ を $\mathbf{x}$ について最小化しても, ヘッセ行列 $\nabla_{\mathbf{xx}}^2 L(\mathbf{x}, \mathbf{u})$ が正定値とは限らないため, 局所最適解が得られない.
- ラグランジュ関数に等式制約 $g_i(\mathbf{x}) = 0$ に対するペナルティ関数を足し合わせた拡張ラグランジュ関数を最小化した後に, ラグランジュ乗数 $\mathbf{u}$ とペナルティ重み ρ を更新する手続きを繰り返す.

$$L_\rho(\mathbf{x}, \mathbf{u}) = f(\mathbf{x}) + \sum_{i=1}^m u_i g_i(\mathbf{x}) + \rho \sum_{i=1}^m g_i(\mathbf{x})^2$$

ペナルティ重み ρ を大きく取れば $\nabla_{\mathbf{xx}}^2 L_\rho(\mathbf{x}, \mathbf{u})$ は正定値

制約つき非線形計画問題

$$\begin{array}{ll} \min. & f(\mathbf{x}) \\ \text{s.t.} & g_i(\mathbf{x}) = 0, \quad i = 1, \dots, m, \\ & \mathbf{x} \in \mathbb{R}^n. \end{array}$$



制約なし非線形計画問題

$$\begin{array}{ll} \min. & f(\mathbf{x}) + \sum_{i=1}^m u_i g_i(\mathbf{x}) + \rho \sum_{i=1}^m g_i(\mathbf{x})^2 \\ \text{s.t.} & \mathbf{x} \in \mathbb{R}^n. \end{array}$$

内点法

- 等式制約つき非線形計画問題の最適性の1次の必要条件に着目し、ニュートン法を用いて条件を満たす解を求める。生成される点列 $\{\mathbf{x}^{(k)}\}$ が全て実行可能領域の内部にあるため内点法と呼ばれる。
- 最適性の1次の必要条件を近似した連立1次方程式*を解いて次の点 $\mathbf{x}^{(k+1)}$ を求めた後に、ペナルティ重み ρ を更新する手続きを繰り返す。

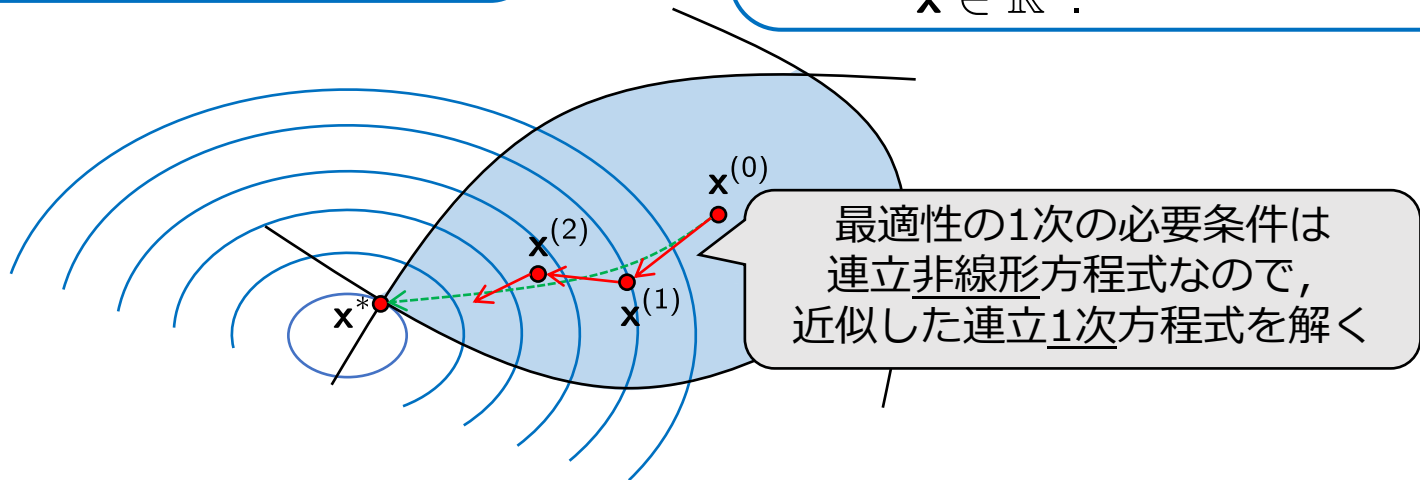
制約つき非線形計画問題

$$\begin{aligned} \min. \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m, \\ & \mathbf{x} \in \mathbb{R}^n. \end{aligned}$$



等式制約つき非線形計画問題

$$\begin{aligned} \min. \quad & f(\mathbf{x}) - \rho \sum_{i=1}^m \log s_i \\ \text{s.t.} \quad & g_i(\mathbf{x}) + s_i = 0, \quad i = 1, \dots, m, \\ & \mathbf{x} \in \mathbb{R}^n. \end{aligned}$$



*凸2次計画問題では近似ではなく連立1次方程式になるので最適解が効率的に求められる。

逐次2次計画法

- 準ニュートン法と同様に, 凸2次計画問題を部分問題として繰り返し解くことで, 不等式制約つき非線形計画問題のKKT条件を満たす点 $\mathbf{x}^*$ に収束する点列 $\{\mathbf{x}^{(k)}\}$ を生成する.
- 有効制約法や内点法を用いて凸2次計画問題を解く.
- ラグランジュ関数 $L(\mathbf{x}, \mathbf{u}) = f(\mathbf{x}) + \sum_{i=1}^m u_i g_i(\mathbf{x})$ のヘッセ行列 $\nabla_{\mathbf{xx}}^2 L(\mathbf{x}, \mathbf{u})$ を近似する正定値対称行列 $\mathbf{B}_k$ を生成する.
- 準ニュートン法と同じBFGS公式を用いると $\mathbf{B}_k$ の正定値性を満たせないので, **パウエルの修正BFGS公式**を用いて $\mathbf{B}_k$ を更新する.

制約つき非線形計画問題

$$\begin{aligned} \min. & f(\mathbf{x}) \\ \text{s.t.} & g_i(\mathbf{x}) \leq 0, i = 1, \dots, m, \\ & \mathbf{x} \in \mathbb{R}^n. \end{aligned}$$



各反復における部分問題

$$\begin{aligned} \min. & \frac{1}{2} \mathbf{d}^\top \mathbf{B}_k \mathbf{d} + \nabla f(\mathbf{x}^{(k)})^\top \mathbf{d} \\ \text{s.t.} & g_i(\mathbf{x}^{(k)}) + \nabla g_i(\mathbf{x}^{(k)})^\top \mathbf{d} \leq 0, i = 1, \dots, m, \\ & \mathbf{d} \in \mathbb{R}^n. \end{aligned}$$

$\mathbf{B}_k$ は正定値

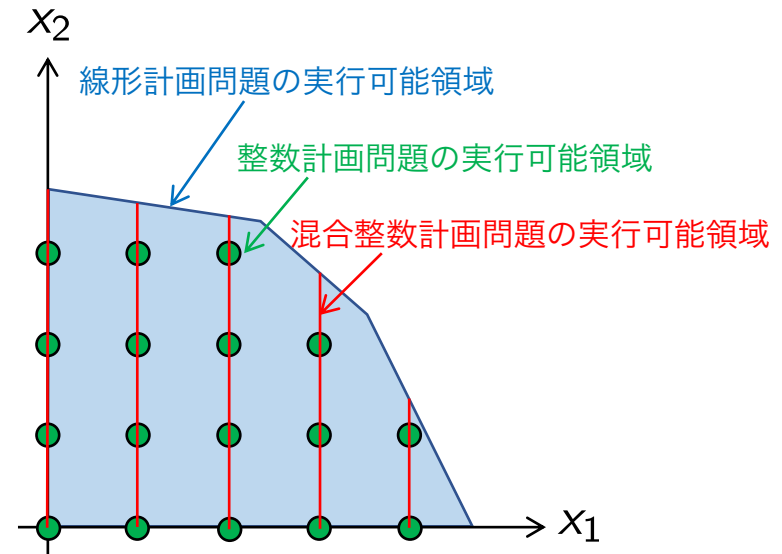
整数計画問題

- 線形計画問題(Linear Program; LP)
 - ✓ 線形制約の下で線形関数を最小化 (最大化)
- 整数計画問題(Integer Program; IP)
 - ✓ 広義：変数に整数条件の付いた最適化問題
 - ✓ 狭義：線形計画問題 + 変数の整数条件

連続変数と整数変数が混在する問題は、**混合整数計画問題(Mixed Integer Program; MIP)**

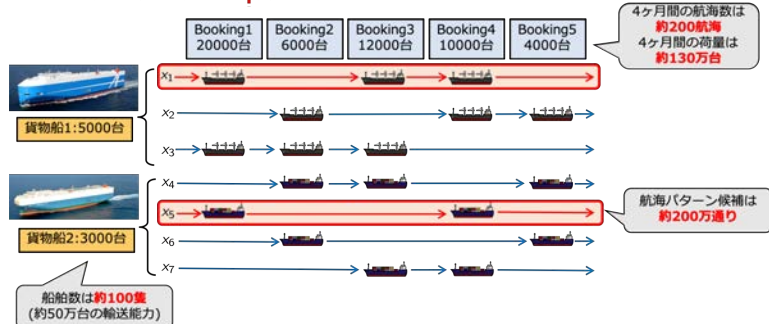
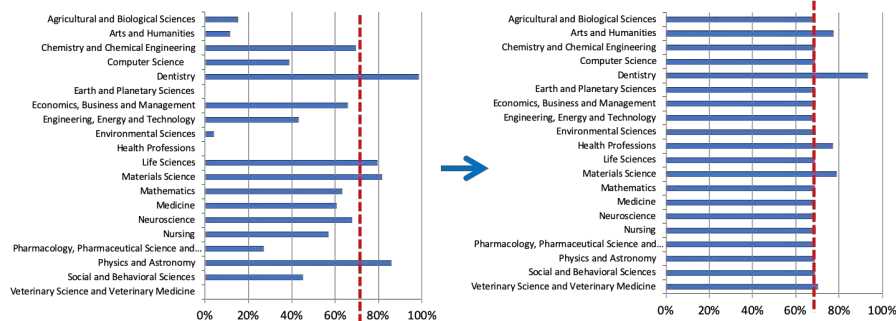
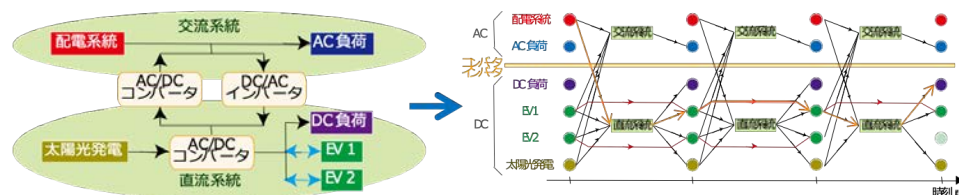
$$\begin{aligned} \min. \quad & \sum_{j=1}^n c_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n a_{ij} x_j \geq b_i, \quad i = 1, \dots, m, \\ & x_j \geq 0, \quad j = 1, \dots, n, \\ & x_j : \text{integer}, \quad j \in J. \end{aligned}$$

J は整数変数の添字集合

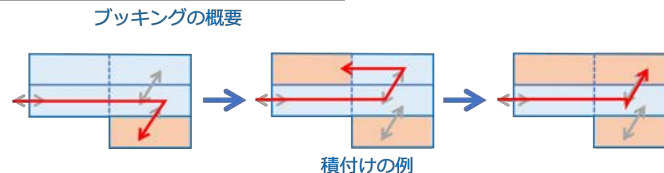


組合せ最適化問題とその応用事例

- **組合せ最適化問題**：最適解を含む解の集合(探索領域)が，集合，順列，割当て，ネットワーク，論理，整数など組合せ的な構造を持つ最適化問題．



寄港順は A→B→C→X→Y→Z	揚げ港		
	X港	Y港	Z港
A港	100	200	2200
積み港 B港	300	2000	300
C港	1300	600	100



多くの現実問題が組合せ最適化問題に定式化できる

組合せ最適化問題の難しさ

- **NP困難問題**：厳密な最適解を求めるのに必要な計算時間が最悪で入力サイズの指数関数になると多くの研究者が考えている問題.
- 都市をちょうど1回ずつ訪問する最短の巡回路を求める巡回セールスマン問題の解候補を列挙すると $(n - 1)!/2$ 通り.



Newsweek, July 26, 1954

都市数	巡回路の総数	計算時間(秒)	
6	60	4.32×10^{-10}	
8	2520	3.23×10^{-8}	
10	1.81×10^5	3.63×10^{-6}	
15	4.36×10^{10}	1.96	
20	6.08×10^{16}	4.87×10^6	約56日
25	3.10×10^{23}	3.88×10^{13}	約122万年
30	4.42×10^{30}	7.96×10^{20}	約25233億年

100TFlopsのコンピュータを用いて
巡回路を列挙したときの計算時間



D15112



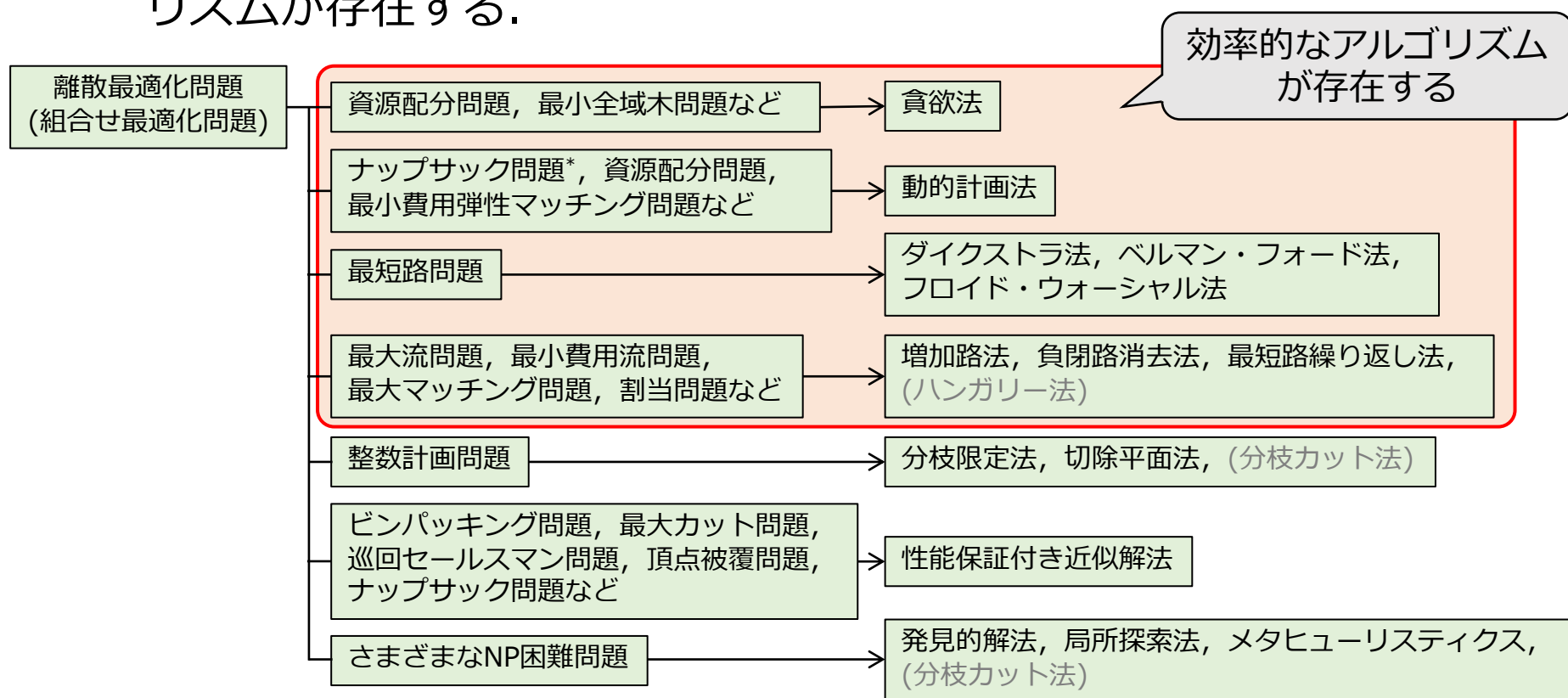
PCB3038

大規模な巡回セールスマン問題を
解く必要が生じることは少なくない

組合せ最適化問題の全ての解を列挙することは非常に困難

効率的に解ける組合せ最適化問題

- ・巡回セールスマン問題や整数計画問題など、多くの組合せ最適化問題はNP困難で効率的なアルゴリズムは期待できない。
- ・特徴的な構造を持つ一部の組合せ最適化問題には効率的なアルゴリズムが存在する。

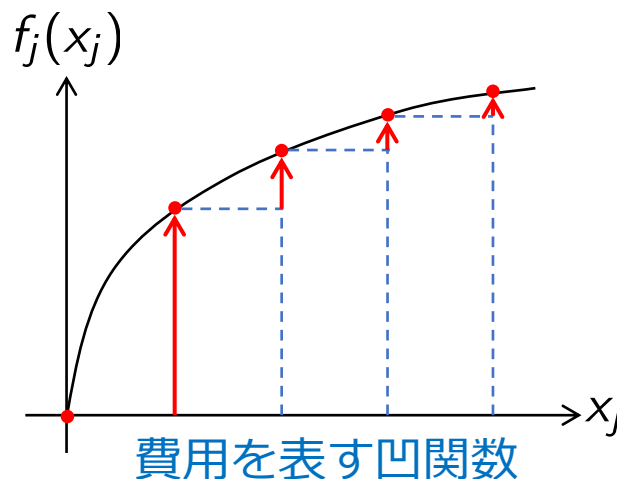


*入力データの各要素(荷物の重さ w_j および袋の容量 C)が定数長の文字列で表現できるという仮定が必要。

資源配分問題に対する貪欲法

- **貪欲法**：局所的な評価値が最も高い要素を選ぶ手続きを繰り返して実行可能解を段階的に構築する手法。
- n 個の事業と利用可能な総資源量 B が与えられる。
- 事業 j に資源を x_j 単位配分すると利益 $f_j(x_j)$ が生じる。
- 総利益を最大化するように資源を各事業に配分するには？
- $f_j(x_j)$ が**凹関数**ならば増分 $d_j(x_j) = f_j(x_j + 1) - f_j(x_j)$ は変数 x_j について非増加*。
- 増分 $d_j(x_j)$ が最大となる変数 x_j を1単位増やす手続きを繰り返す。

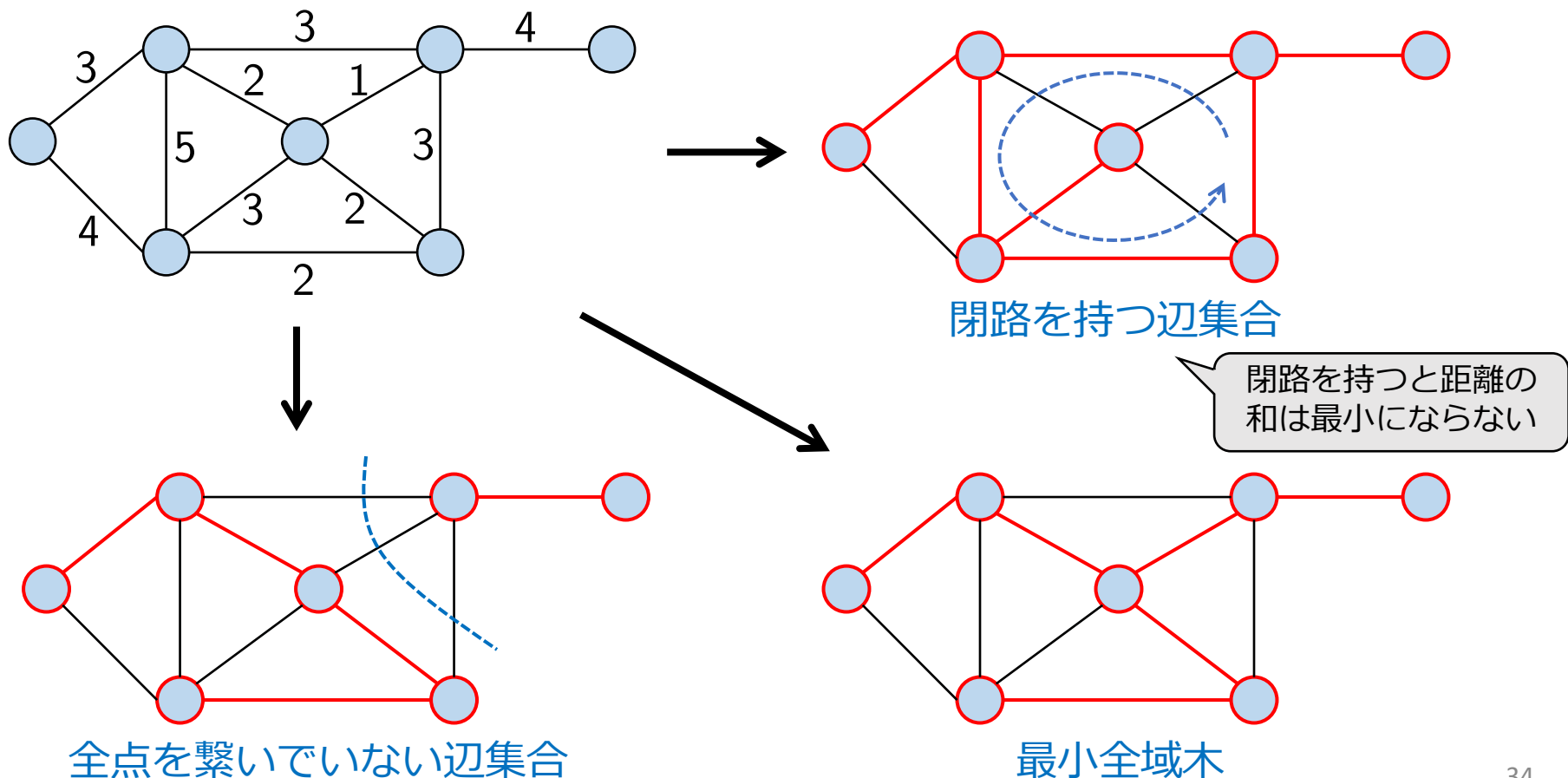
$$\begin{aligned} \max. \quad & \sum_{j=1}^n f_j(x_j) \\ \text{s.t.} \quad & \sum_{j=1}^n x_j = B, \\ & x_j : \text{integer}, \quad j = 1, \dots, n. \end{aligned}$$



*例えば、同じ広告を繰り返し配信すると広告1回当たりの効果は徐々に減少する(限界効用逓減の法則)。

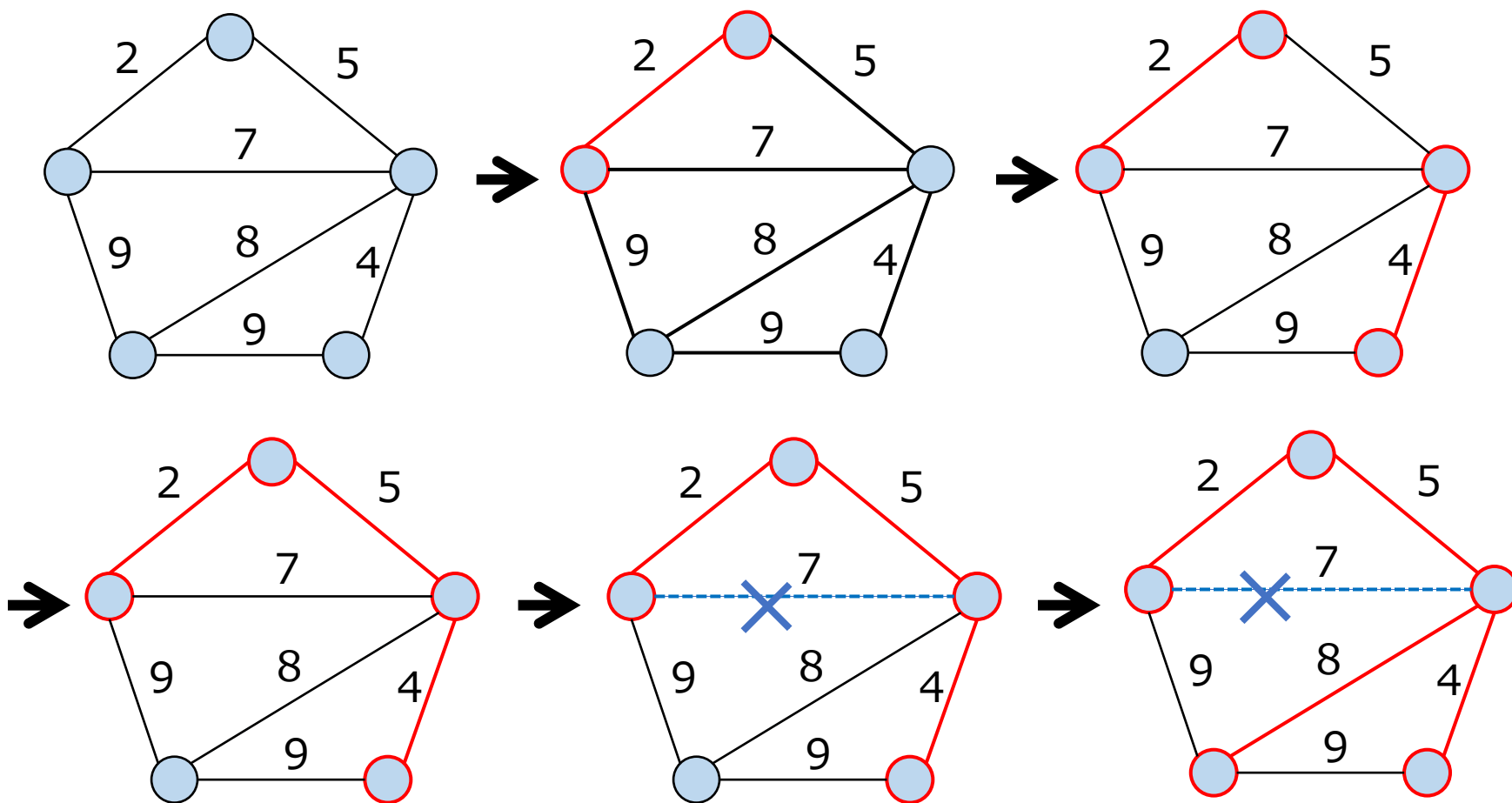
最小全域木問題に対する貪欲法

- グラフ $G = (V, E)$, 各辺 $e \in E$, の長さ $d_e > 0$ が与えられる.
- グラフ G の全ての頂点をつなぐ最短の辺の部分集合 $T \subseteq E$ は?
- クラスカル法, プリム法など効率的なアルゴリズムが存在する.



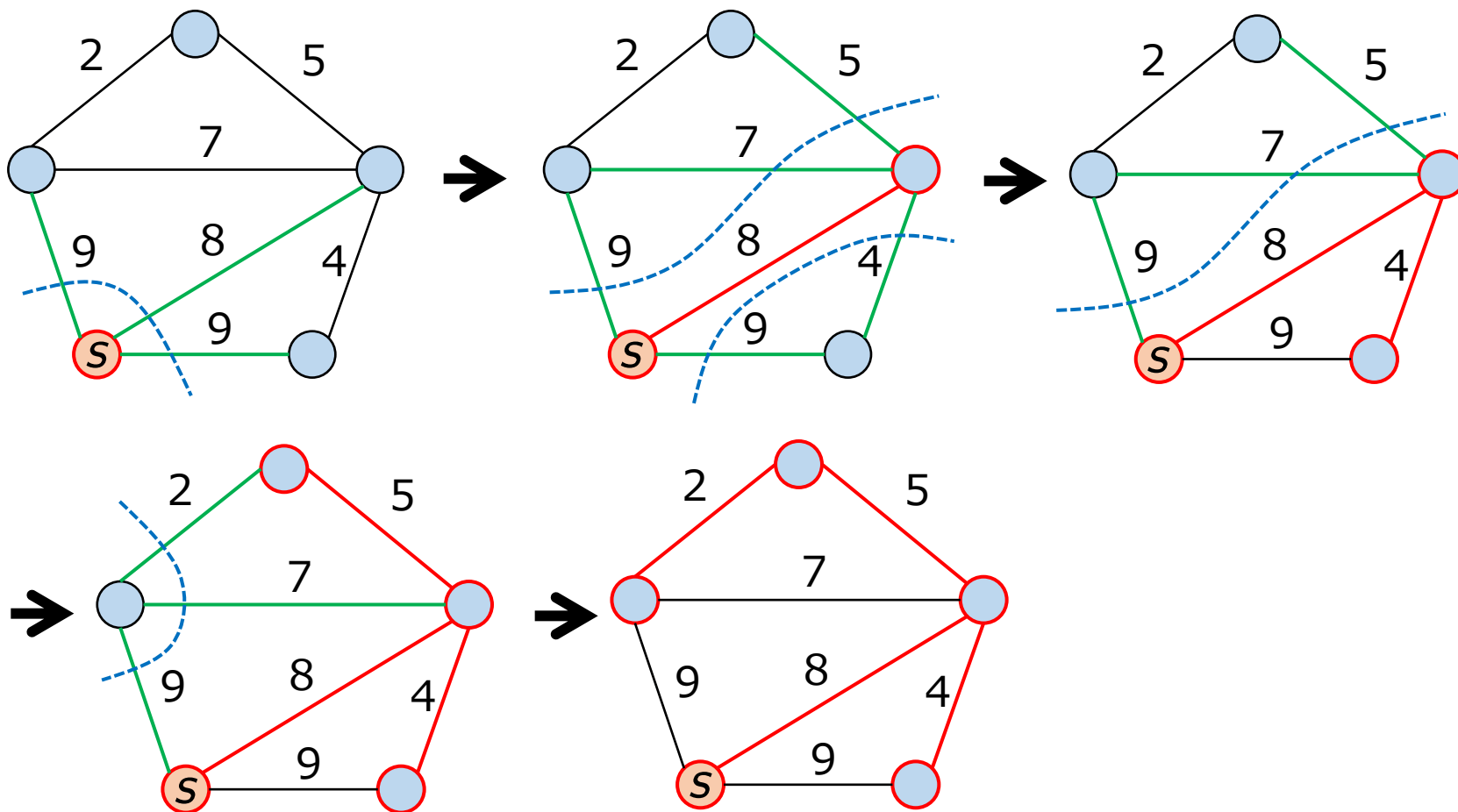
クラスカル法

- 長さの小さい順に辺を追加する.
- 各反復では閉路ができないように長さ最小の辺を追加する.



プリム法

- 適当な点 s を根とし辺を1本ずつ追加して木 T を成長させる.
- 各反復では点 $u \in T$ と点 $v \notin T$ をつなぐ長さ最小の辺を追加する.



資源配分問題に対する動的計画法

- **動的計画法**：自明な解を持つ部分問題から始めて，順次大きな部分問題を解く．重複する部分問題を繰り返し解かないように履歴を表などに記録する．
- $f_j(x_j)$ が**凹関数とは限らない**資源配分問題について，資源の総量を $u (< B)$ に，事業の候補を $1, \dots, k (< n)$ に制限した部分問題の最適値を $f(k, u)$ とする．
- $f(k, u)$ は以下の漸化式で求められる．

$$\begin{cases} f(1, u) = f_1(u) \\ f(k, u) = \max\{f(k-1, u-x_k) + f_k(x_k) \mid x_k = 0, \dots, u\} \end{cases}$$

x_j のみを変数に持つ関数(**分離型**)

$$\begin{aligned} \max. & \quad 10|x_1 - 1| + x_2^2 + 2(x_3 - 1)^2 + \frac{60}{2x_4 - 9} \\ \text{s.t.} & \quad x_1 + x_2 + x_3 + x_4 = 6, \\ & \quad x_1, x_2, x_3, x_4 : \text{integer.} \end{aligned}$$

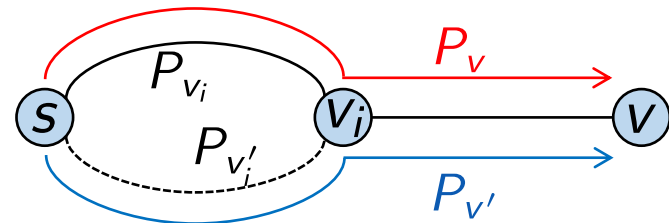
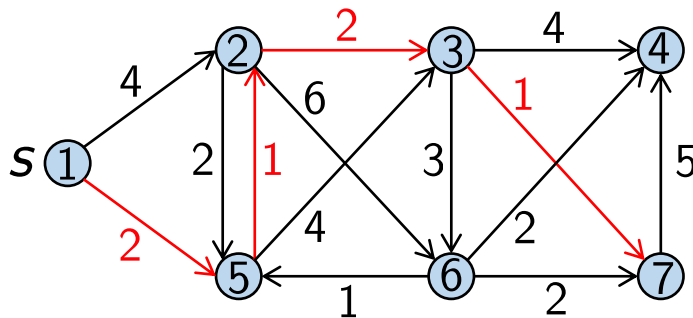
事業

資源

	0	1	2	3	4	5	6
1	10	0	10	20	30	40	50
2	10	11	14	20	30	40	50
3	12	13	16	22	32	42	60
4	$\frac{56}{3}$	$\frac{144}{7}$	24	32	72	73	76

最短路問題に対するラベリング法

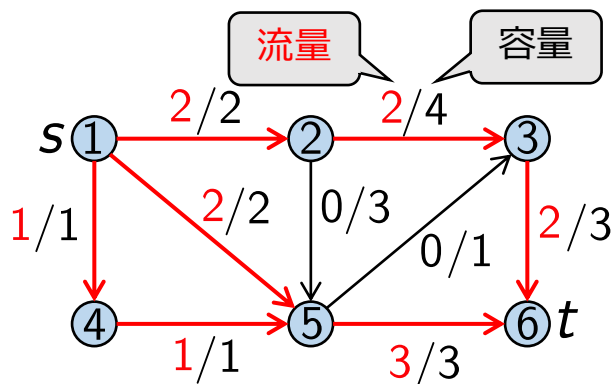
- グラフ $G = (V, E)$, 各辺 $e \in E$ の長さ $d_e > 0$ が与えられる.
- 始点 $s \in V$ から各頂点 $v \in V$ への最短路は? (単一起点最短路問題)
- **部分構造最適性**: 始点 s から頂点 v への路 P_v の部分路 P_{v_i} は, 始点 s から頂点 v_i への最短路.
- 始点 s から各頂点 $v \in V$ への最短路の長さの上界を表すラベル値 f_v を更新する手続きを繰り返す.
- **ダイクストラ法**: 各反復でこれまでに選ばれていない頂点の中で最小のラベル値 f_v を持つ頂点を選ぶ手続きを繰り返す.
- **ベルマン・フォード法**: 全ての頂点を適当な順番で1回ずつ選びラベル値 f_v を更新する手続きを繰り返す.



部分構造最適性

最大流問題に対する増加路法

- 有向グラフ $G = (V, E)$ の各辺 $e \in E$ の容量 $u_e > 0$ が与えられる。
- 入口 s から出口 t に流れるフローの総量を最大にするには？
- **増加路法**：残余ネットワークの入口 s から出口 t にいたる増加路を見つけ、増加路に沿って流すフローを増加する手続きを繰り返す。



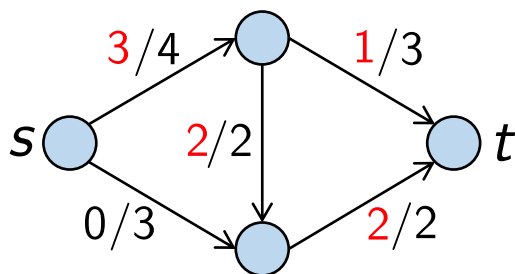
max. $\sum_{e \in \delta^+(s)} x_e$

s.t. $\sum_{e \in \delta^+(v)} x_e - \sum_{e \in \delta^-(v)} x_e = 0, \quad v \in V \setminus \{s, t\},$

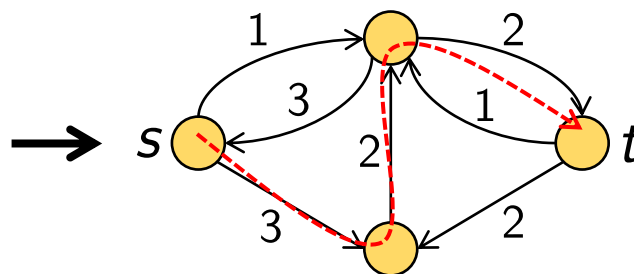
$0 \leq x_e \leq u_e, \quad e \in E.$

線形計画問題に定式化できる

x_e : 辺 $e \in E$ の流量(変数)



実行可能なフロー

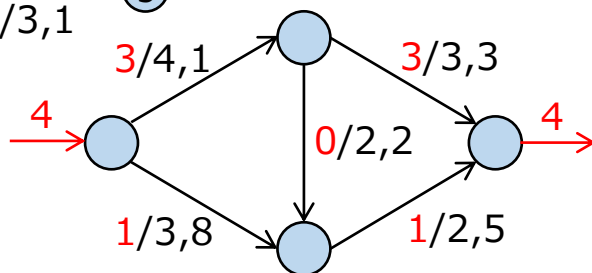
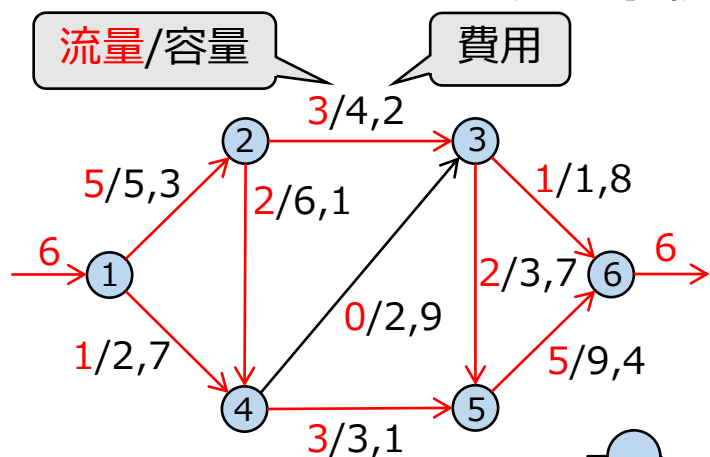


残余ネットワーク

* $\delta^+(v), \delta^-(v)$ はそれぞれ頂点 v を始点, 終点とする辺集合。

最小費用流問題に対する負閉路消去法

- 有向グラフ $G = (V, E)$ の各辺 $e \in E$ の容量 $u_e > 0$, 費用 c_e と各頂点 $v \in V$ から流出するフローの量 b_v が与えられる*.
- 各頂点の流出(流入)量を満たし総費用を最小にするフローは?
- 負閉路消去法**: 残余ネットワークの負閉路を見つけ, 負閉路に沿ってフローを流す手続きを繰り返す.

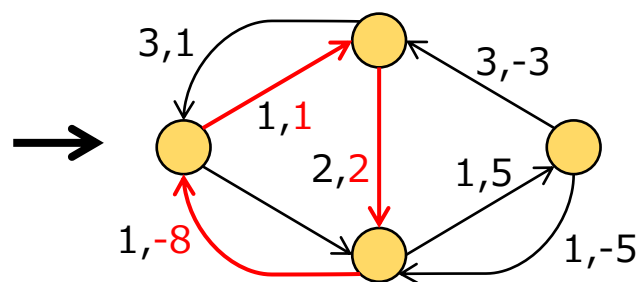


実行可能なフロー

線形計画問題に定式化できる

$$\begin{aligned} \min. \quad & \sum_{e \in E} c_e x_e \\ \text{s.t.} \quad & \sum_{e \in \delta^+(v)} x_e - \sum_{e \in \delta^-(v)} x_e = b_v, \quad v \in V, \\ & 0 \leq x_e \leq u_e, \quad e \in E. \end{aligned}$$

x_e : 辺 $e \in E$ の流量(変数)



残余ネットワーク

* $\sum_{v \in V} b_v = 0$ を満たすと仮定する.

割当問題

- m 人の学生を n 個のクラスに割り当てる．ただし，各クラスの受講者には上限が与えられる．学生の満足度を最大にするクラスの編成は？
- 割当問題は最小費用流問題に変換できる．
- ハンガリー法など効率的なアルゴリズムが存在する．

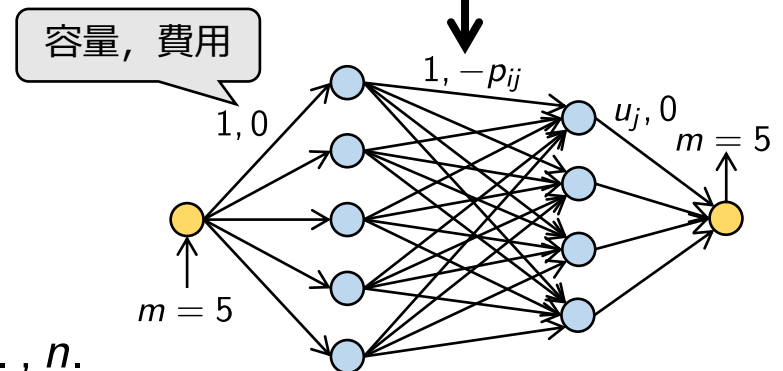
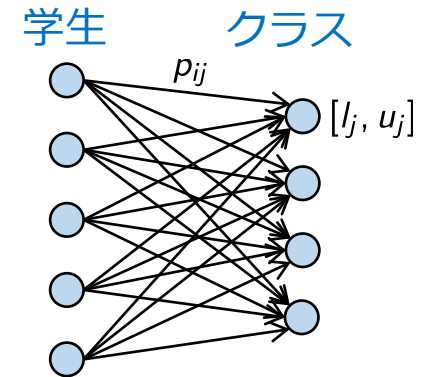
u_j : クラス j の受講者数の上限(定数)

p_{ij} : 学生 i のクラス j に対する満足度(定数)

x_{ij} : 学生 i をクラス j に割り当てる(変数)

$$\begin{aligned} \max \quad & \sum_{i=1}^m \sum_{j=1}^n p_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{i=1}^m x_{ij} \leq u_j, \quad j = 1, \dots, n, \\ & \sum_{j=1}^n x_{ij} = 1, \quad i = 1, \dots, m, \\ & x_{ij} \in \{0, 1\}, \quad i = 1, \dots, m, \quad j = 1, \dots, n. \end{aligned}$$

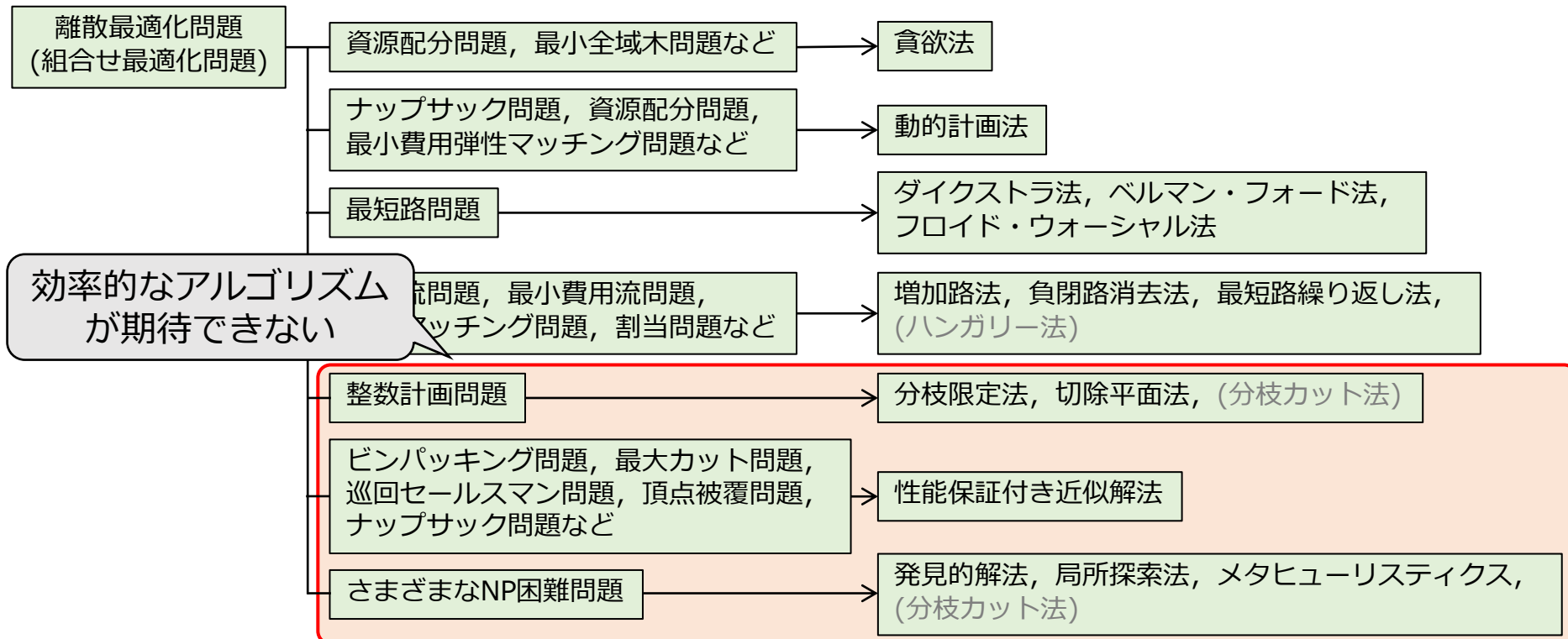
線形計画問題に定式化できる*



*変数 x_{ij} の整数条件を $0 \leq x_{ij} \leq 1$ に緩和した線形計画問題に単体法を適用すると整数解が得られる。

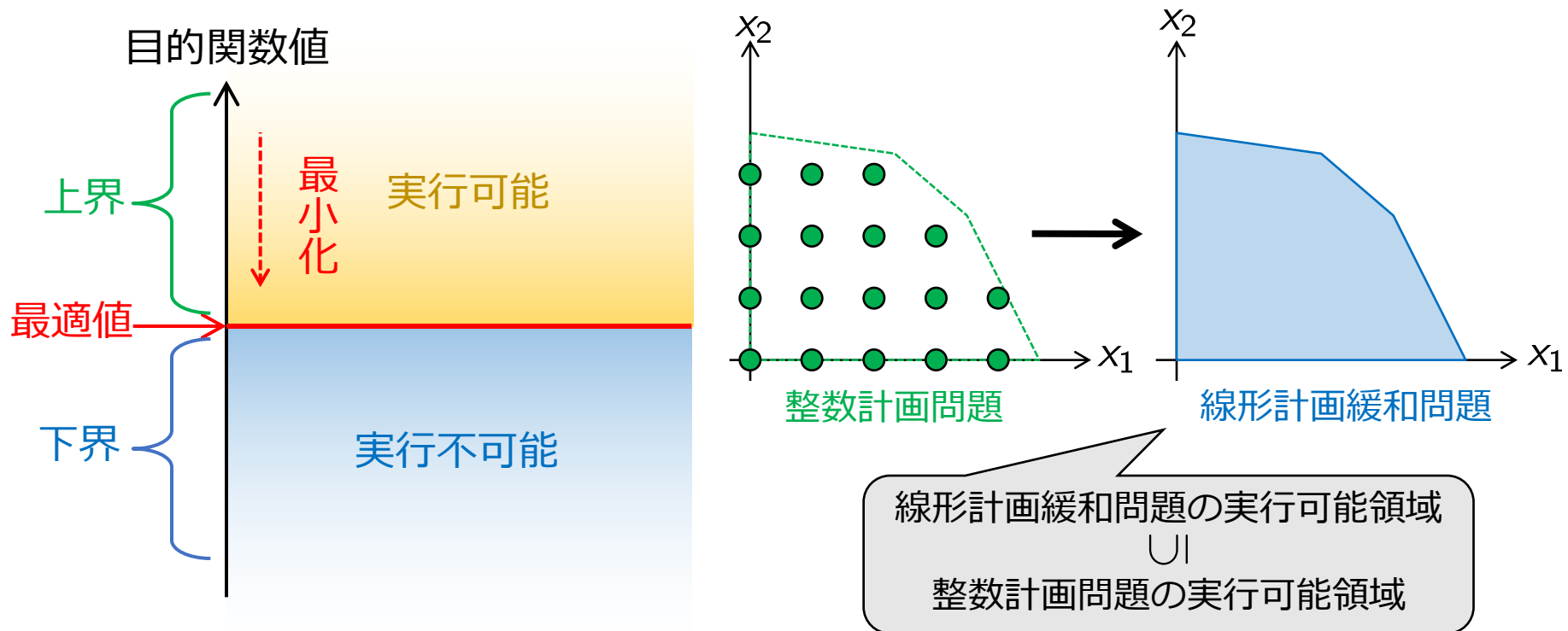
計算困難な組合せ最適化問題

- **厳密解法**：任意の入力データに対して最適解を1つ出力するアルゴリズム。
- **近似解法**：任意の入力データに対して最適値に対する近似性能を保証する実行可能解を1つ出力するアルゴリズム。
- **発見的解法**：最適値に対する近似性能が保証されていない実行可能解を1つ出力するアルゴリズム。



整数計画問題を解く

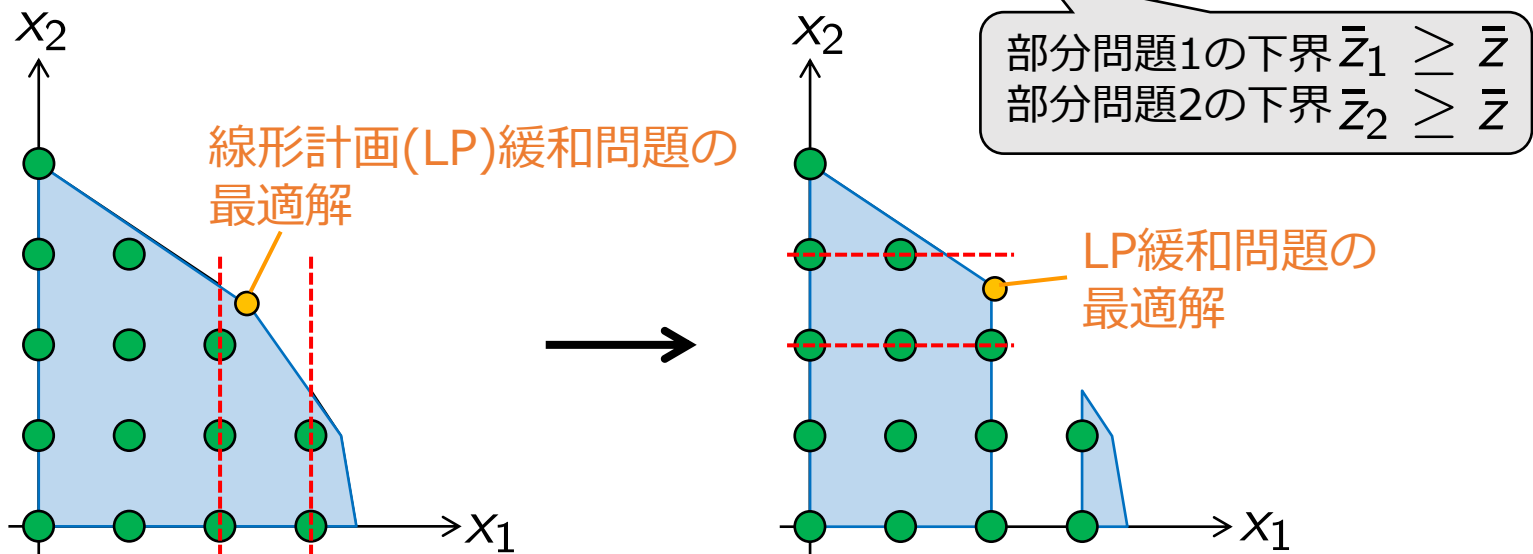
- 計算困難な最適化問題では最適値が簡単に求まらない.
- 最適値の**上界**と**下界**を求めて最適値の取り得る範囲を限定する.
- **緩和問題**：原問題の一部の制約条件を緩めた問題
 - ✓ 整数変数の整数条件を外す → **線形計画緩和計画問題**



分枝限定法

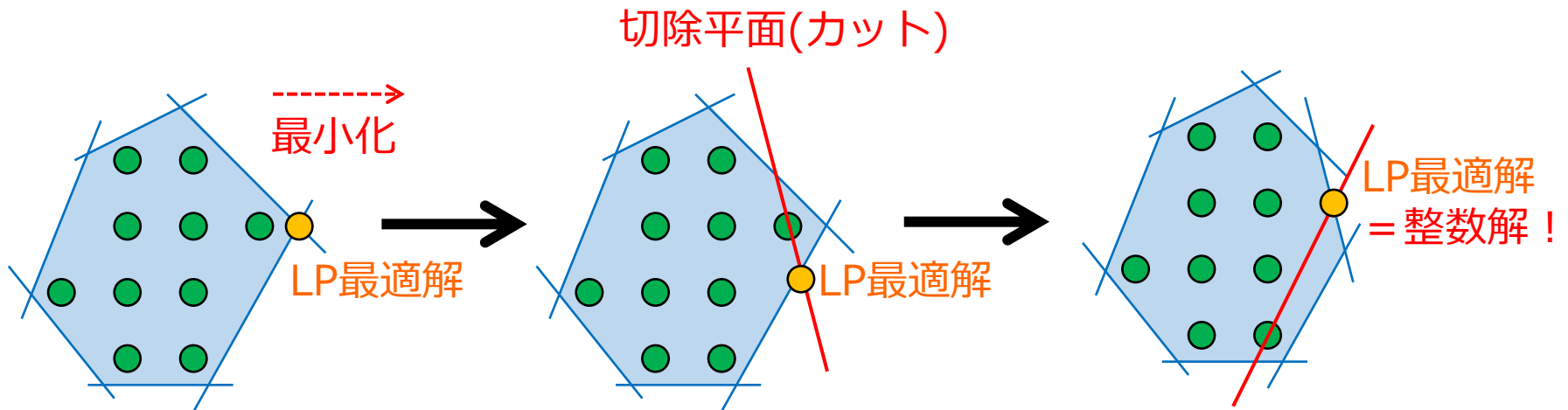
- 最適化問題に対する汎用的な厳密解法.
- 分枝操作: 実行可能領域を分割して部分問題を生成する.
(緩和解が実数値を取る変数 x_t を選んで, $x_t \leq \lfloor \bar{x}_t \rfloor$ を追加した問題と, $x_t \geq \lceil \bar{x}_t \rceil$ を追加した問題に分割する)
- 限定操作: 緩和問題から得られる下界値を用いて最適解が得られる見込みのない部分問題を省く.

(分枝前の下界値 $\bar{z} \leq$ 分枝後の下界値 $\min\{\bar{z}_1, \bar{z}_2\}$)



切除平面法

- 全ての実行可能な整数解を含む凸包は凸多面体 → 線形計画問題
- 前もって凸包の不等式表現を知ることは困難だしその数も膨大.
- 線形計画問題の最適解 $\bar{x}$ が整数解でないとき, $\bar{x}$ を切除する制約式を追加する (ただし, 整数計画問題の実行可能解は切除しない).
- 切除平面を追加するに従って最適解に近づく.



整数計画ソルバーは分枝限定法と切除平面法を組み合わせた
分枝切除平面法を用いている

性能保証付き近似解法

- 同時に緩和問題を解いて下界も求めるアルゴリズムを設計できれば、任意の入力データに対して性能を保証できる場合もある。
- 最小化問題 Q の入力データ $I \in Q$ の最適値を $OPT(I)$ 、アルゴリズム A を適用して得られた解の目的関数の値を $A(I)$ とする。
- **近似比率**：最小化問題 Q の任意の入力データ $I \in Q$ に対して、

$$A(I) \leq r OPT(I)$$

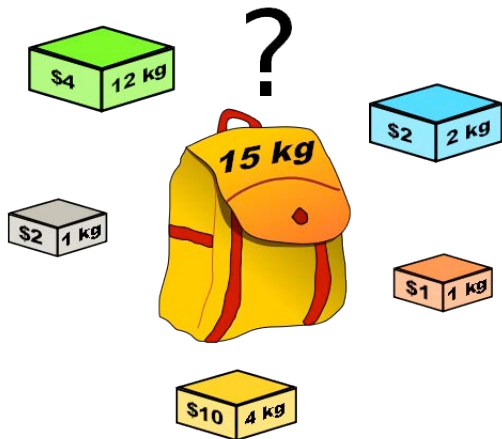
が成り立つとき、アルゴリズム A は近似比率 r を持つと呼ぶ。

- NP困難問題は以下の3つのクラスに分類できる。
 - ✓ どんなに小さな正定数 ε に対しても $(1 + \varepsilon)$ -近似解法が存在する。
 - ✓ ある正定数 ε に対して $(1 + \varepsilon)$ -近似解法は存在するが、 ε に下限がある。
 - ✓ 定数近似解法が存在しない*。

*正確には、定数近似アルゴリズムが存在すればNP完全問題が多項式時間で解けてしまう。

ナップサック問題の線形計画緩和問題

- 1つの袋と n 個の荷物が与えられる。袋に詰め込める荷物の重さ合計の上限を C ，荷物 j の重さを w_j ，価値を p_j とする。
- 価値の合計が最大となる荷物の詰め合わせは？
- ナップサック問題の線形計画(LP)緩和問題は容易に解ける。



$$\begin{aligned} \max. \quad & \sum_{j=1}^n p_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n w_j x_j \leq C, \\ & x_j \in \{0, 1\}, \quad j = 1, \dots, n. \end{aligned}$$

x_j : 荷物 j の袋への詰め込み(変数)

荷物を $\frac{p_1}{w_1} \geq \frac{p_2}{w_2} \geq \dots \geq \frac{p_n}{w_n}$ と並べ直すとLP緩和解は以下の通り

$$\bar{x}_j = \begin{cases} 1 & j = 1, \dots, k \\ \frac{C - \sum_{j=1}^k w_j}{w_{k+1}} & j = k + 1 \\ 0 & j = k + 2, \dots, n \end{cases}$$

荷物を単位重さ当たりの価値の降順に詰め込み、はみ出した荷物を切る

$\left(\sum_{j=1}^k w_j \leq C, \sum_{j=1}^{k+1} w_j > C \right)$

ナップサック問題の1/2-近似解法

- 下記の2通りの解の目的関数の値が良い方を $A(I)$ とする.
 - ✓ LP緩和解から $(k + 1)$ 番目の荷物を除いて得られる解 → 貪欲法
 - ✓ 最も価値の高い荷物を1つだけ詰めて得られる解*
- LP緩和解の最適値を $LP(I)$ とすると $LP(I) \geq OPT(I)$

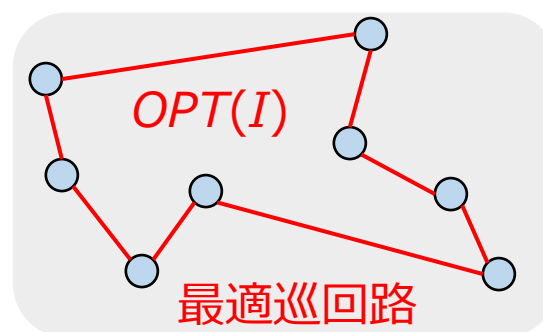
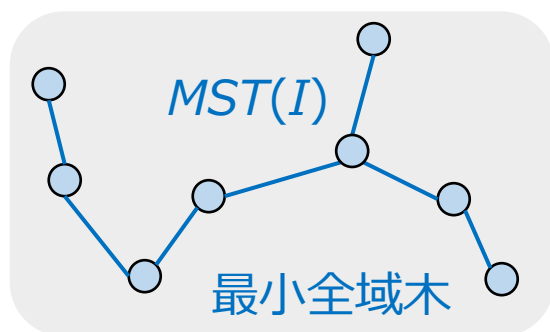
$$\begin{aligned} LP(I) &= \sum_{j=1}^k p_j + p_{k+1} \left(\frac{C - \sum_{j=1}^k w_j}{w_{k+1}} \right) \\ &\leq A(I) + \max_{j=1, \dots, n} p_j \\ &\leq 2A(I) \end{aligned}$$

$$\text{以上より } A(I) \geq \frac{1}{2} LP(I) \geq \frac{1}{2} OPT(I)$$

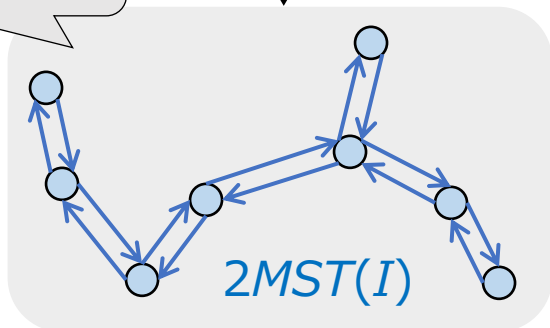
*袋に空きがあれば他の荷物を追加で詰め込んでも構わない。

巡回セールスマン問題の2-近似解法

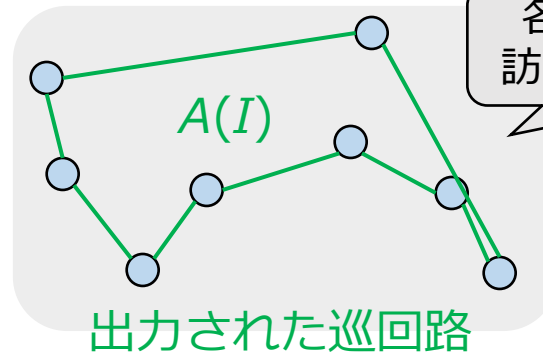
- 貪欲法を用いて最小全域木を求める(クラスカル法, プリム法など).
- 最小全域木に沿って全ての都市を巡回する*.
- 一度訪れた都市は飛ばして全ての都市を一度だけ訪問するように巡回路を繋ぎ替える.



最小全域木に沿って
全ての都市を訪問



$$A(I) < 2MST(I) < 2OPT(I)$$



各都市を一度ずつ
訪問するように修正

*同じ都市を複数回訪れても構わないので正確には巡回路ではない.

発見的解法

- ヒューリスティクス(発見的解法)

- ✓ もともとは発見的法則，経験則に基づく問題解決の方法を指す。それが転じて，最適化問題における最適性の保証のないアルゴリズムの総称と呼ばれる。

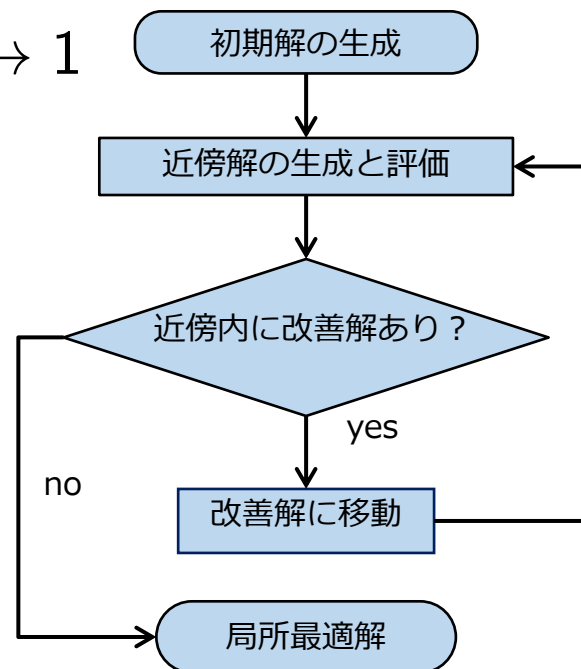
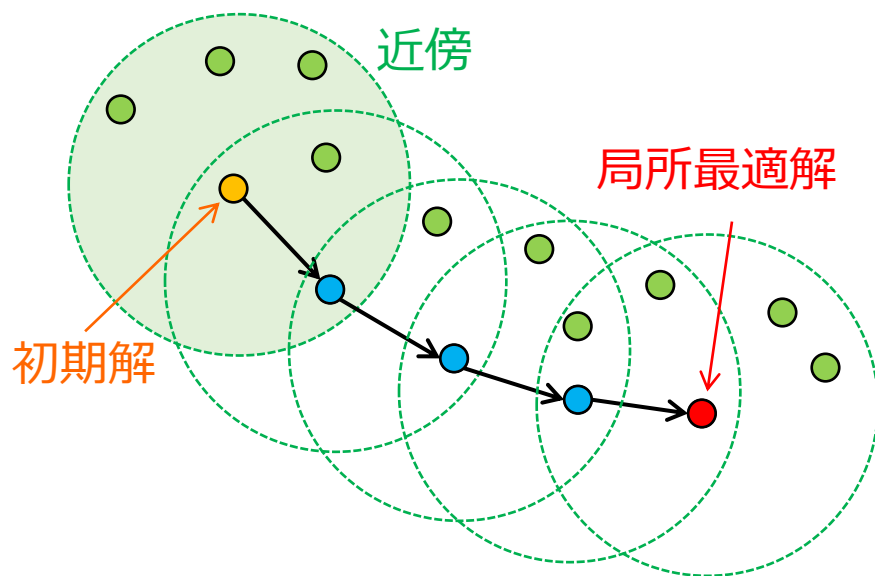
- メタヒューリスティクス

- ✓ 最適化問題に対する実用的なアルゴリズムを設計するための一般的な枠組み(レシピ集)，もしくはそのような考え方に従って設計された様々なアルゴリズムの総称と呼ばれる。
- ✓ 貪欲法や局所探索法などの基本的な手法に様々な手法を組み合わせ，探索の**集中化**と**多様化**をバランス良く実現する。
- ✓ アニーリング法，タブー探索法，遺伝的アルゴリズムを始めとして多くの探索戦略が提案されている。

自然現象にアナロジーを持つ手法が多いが，自然現象を模倣すればアルゴリズムの性能が良くなるというわけではない。

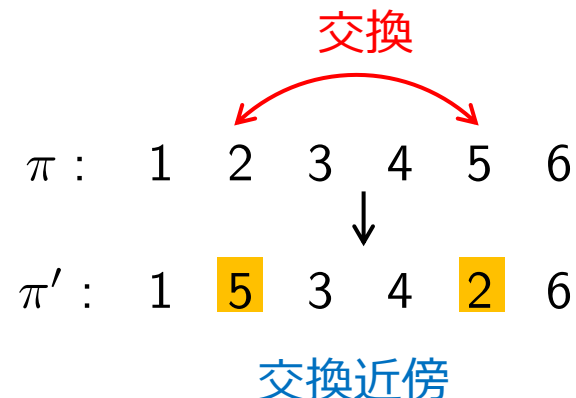
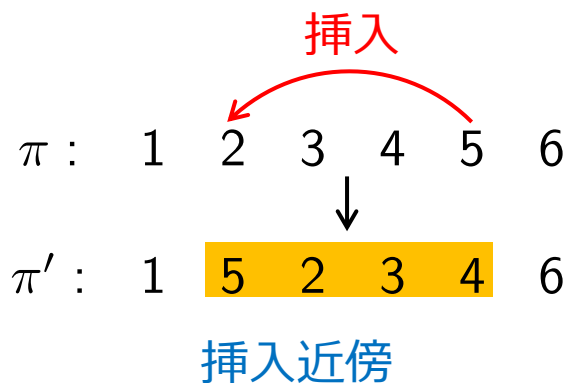
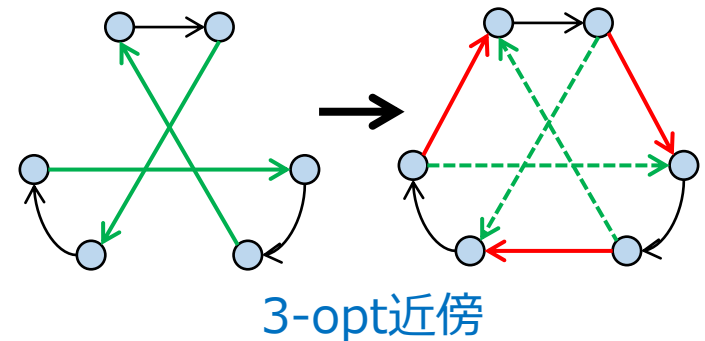
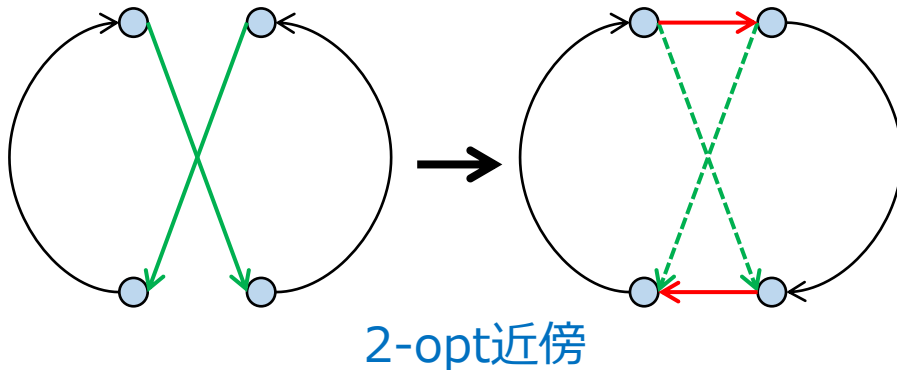
局所探索法

- 適当な初期解 $\mathbf{x}$ から始めて、現在の解 $\mathbf{x}$ の近傍 $\mathbf{x}' \in \text{NB}(\mathbf{x})$ に改善解があれば移動する(近傍探索).
- 勾配法では微分 $\nabla f(\mathbf{x})$ を用いるが、局所探索法では差分 $\Delta f(\mathbf{x}) = f(\mathbf{x}') - f(\mathbf{x})$ を用いて探索方向を決定する.
- 0-1整数計画問題の近傍の例
 - ✓ 1-flip近傍: $x_j = 0 \rightarrow 1$ (or $1 \rightarrow 0$)
 - ✓ 2-flip近傍: $x_{j_1} = 1 \rightarrow 0, x_{j_2} = 0 \rightarrow 1$



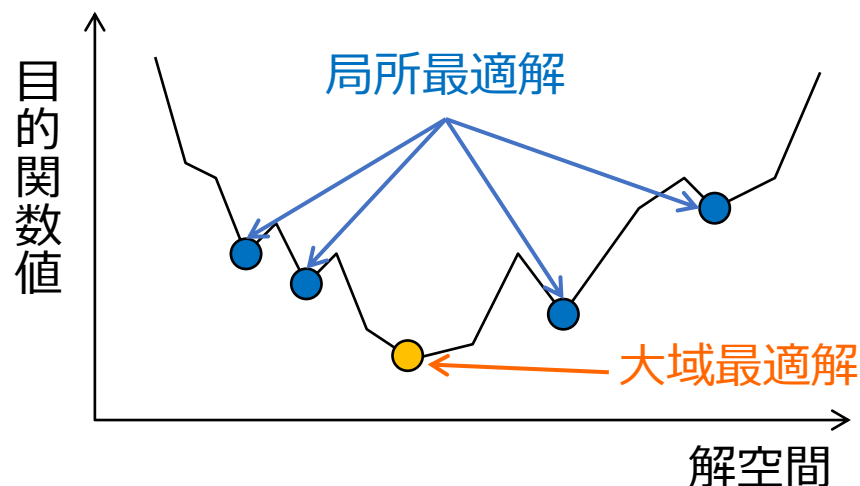
局所探索法

- 組合せ最適化問題の構造に合わせた近傍が設計できる.
- 巡回セールスマン問題では, 巡回路の辺を k 本入れ替える k -opt近傍が良く知られる.
- 解が順列で表される場合は挿入近傍や交換近傍が良く使われる.



メタヒューリスティクスの戦略

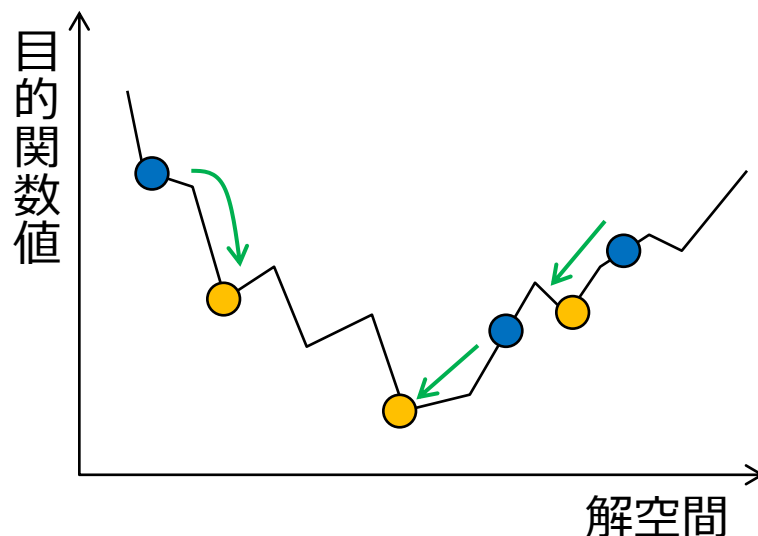
- 多くの最適化問題は**近接最適性**(良い解の近くに良い解がある)と**多峰性**(局所最適解が多数存在する)を持つ。
- 近似解法の基本戦略は**貪欲法**(構築法)と**局所探索法**(改善法)。
- 局所探索法を1回適用しただけでは精度の悪い局所最適解しか得られないので、探索の集中化と多様化を実現する手法を導入する。
- **集中化**：過去の探索で得られた良い解に似通った構造を持つ解を集中的に探索する。
- **多様化**：探索が狭い領域で停滞するのを避けるために、過去の探索で得られた解と異なる構造を持つ解を積極的に探索する。



多スタート局所探索法

複数の初期解に局所探索法を適用

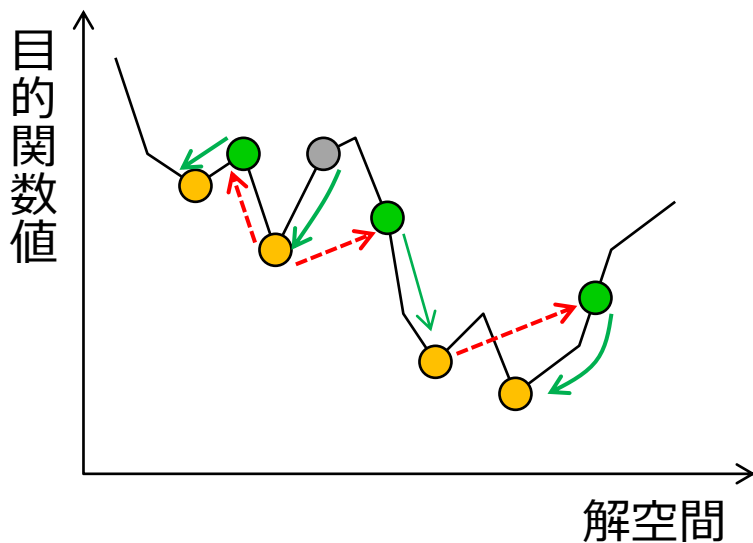
- ランダムに生成した複数の初期解から局所探索法を実行し、得られた解の中で最良のものを出力する。
- 各試行が独立なため、局所探索法の適用回数を増やしてもより良い解が得られる効果は急速に小さくなる。
- ランダム性を加えた貪欲法実行して初期解を生成する手法は **GRASP**(Greedy Randomized Adaptive Procedure)と呼ばれる。



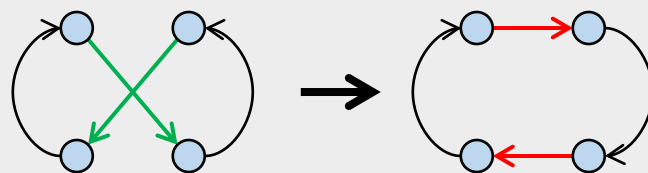
反復局所探索法

複数の初期解に局所探索法を適用
過去の探索で得られた良い解に変形を加える

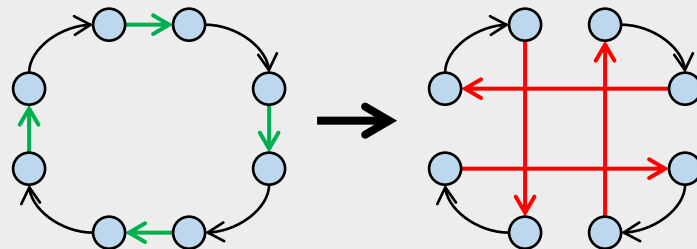
- 過去の探索で得られた良い解にランダムな変形を加えたものを初期解として局所探索法を適用する手続きを繰り返す。
- 逆戻りを防ぐために、局所探索法とは異なる近傍操作を用いて解に変形を加えることが望ましい。
- 変形時に用いる近傍の大きさを適応的に変化させる可変近傍探索法も知られる。



局所探索では2-opt近傍



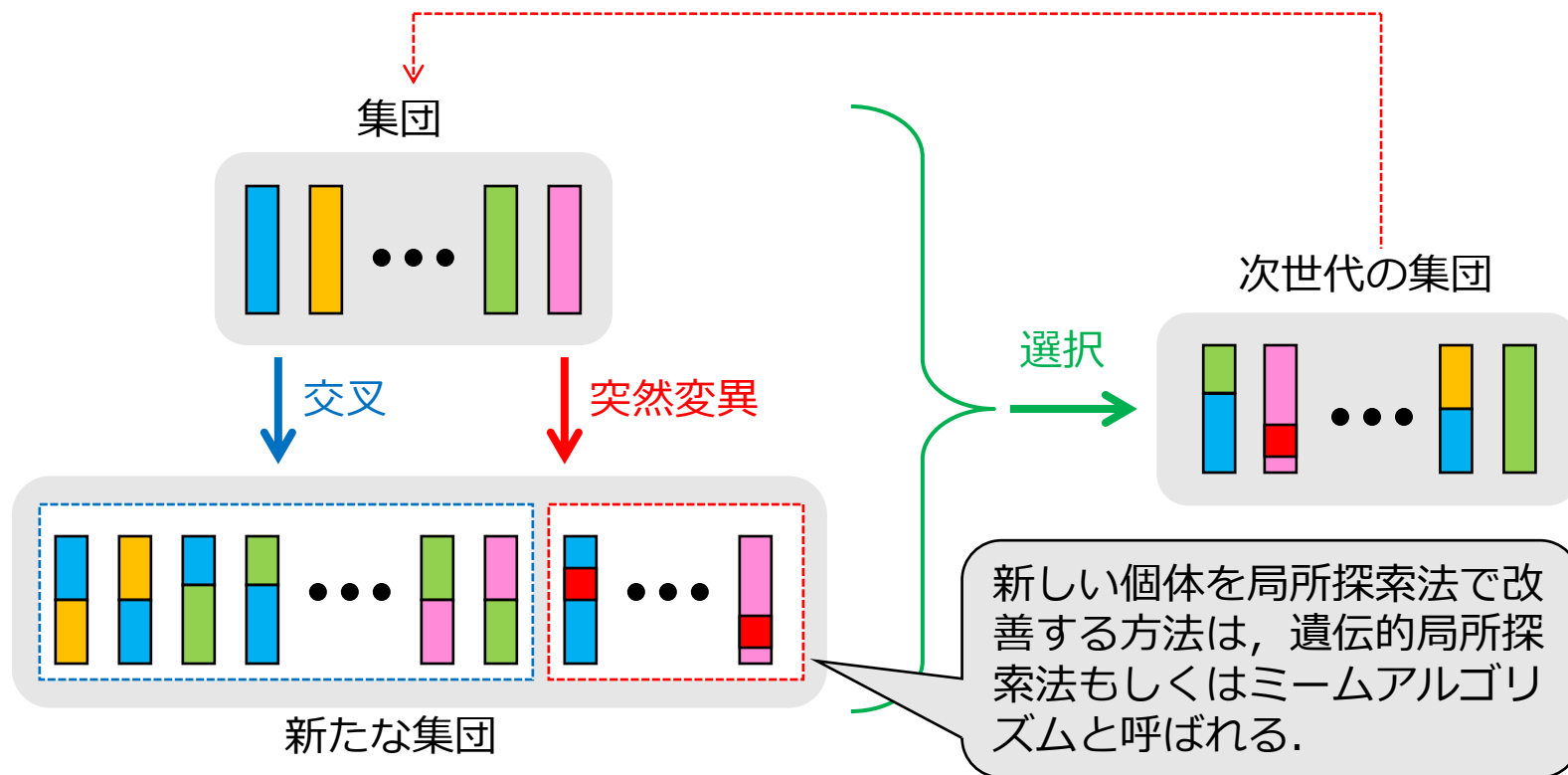
変形にdouble bridge近傍



遺伝的アルゴリズム

複数の初期解に局所探索法を適用
過去の探索で得られた良い解に変形を加える

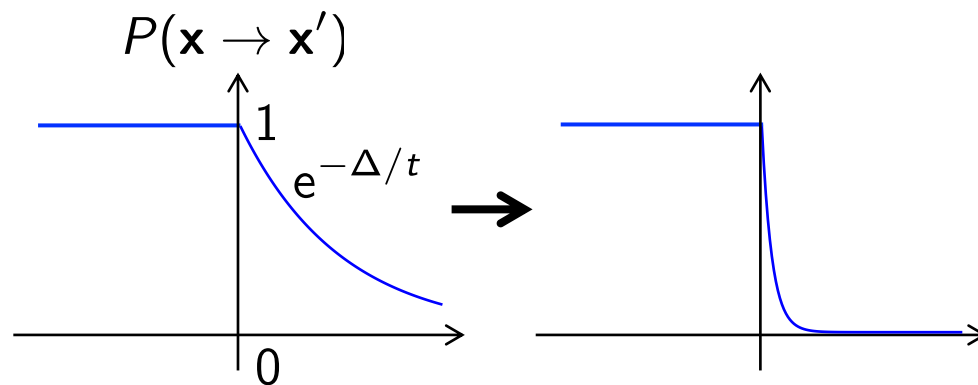
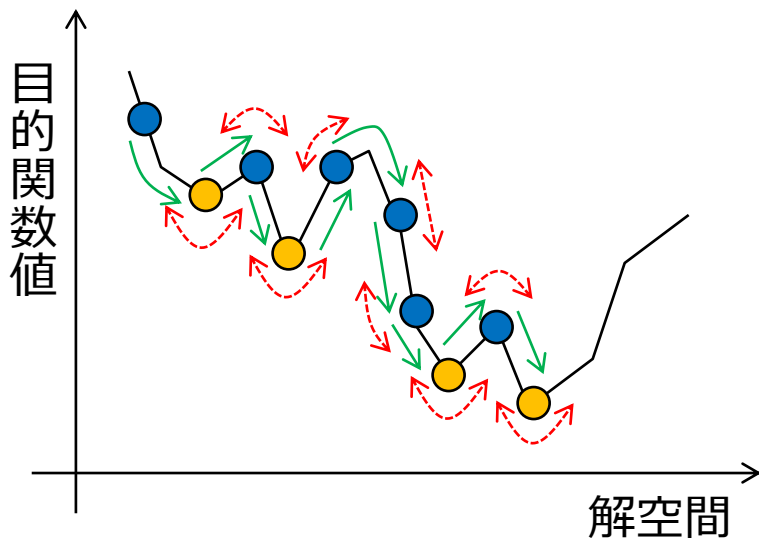
- 複数の解を集団として保持することで探索の多様化を実現する。
- 交叉や突然変異などの操作を適用して新たな解を生成し，選択により次世代の集団を決定する一連の手続きを繰り返し適用する。
- より自由な枠組みに基づいて複数の解から新たな解を生成する**散布局所探索法**や**パス再結合法**などが知られる。



アニーリング法

改悪解への移動を許容して局所探索法を再開

- 現在の解の近傍からランダムに選んだ解が改善解ならば移動し、改悪解ならば目的関数の値の変化量に応じた確率で移動することで、質の悪い局所最適解から脱出し、より良い解を探索する。
- 物理現象の焼きなましにアイデアを借りて、温度と呼ばれるパラメータ t により改悪解に移動する確率を適応的に調整する。
- 探索の開始時にはランダムな移動が生じ易いように温度を高め設定し、探索が進むにつれて徐々に温度を低くする。

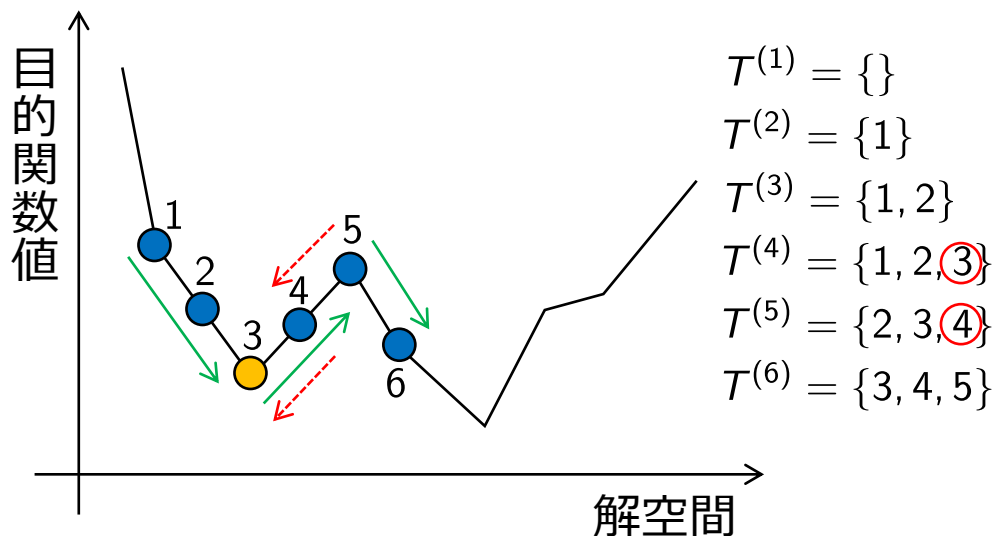


アニーリング法の解の移動確率

タブー探索法

改悪解への移動を許容して局所探索法を再開
探索の履歴に応じて探索方向を制御

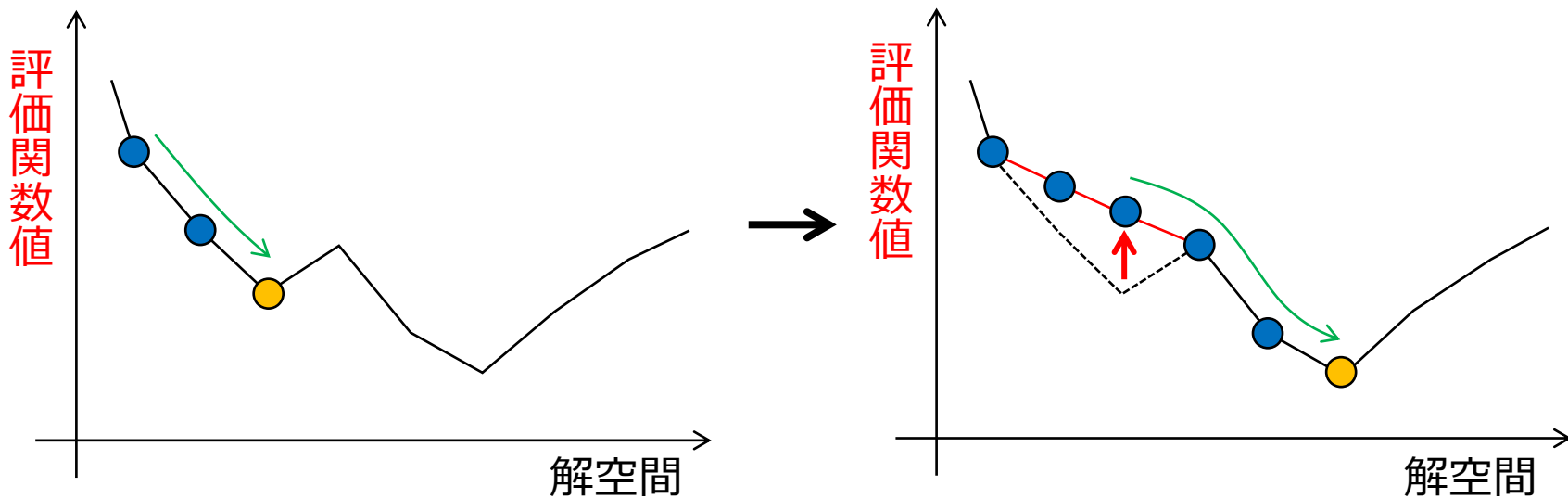
- 現在の解を除く近傍内の最良解に移動する手続きを繰り返すことで局所最適解から脱出する.
- 最近に探索した解に戻る巡回を防ぐために, タブーリストと呼ばれる移動を禁止する解の集合を用意し, 現在の解とタブーリスト T を除く近傍内の最良解に移動する.
- 最近に探索した解そのものではなく, 近傍操作により変化した変数(とその値の組)をタブーリストに記憶することが多い.



誘導局所探索法

目的関数と異なる評価関数の下で局所探索法を実行
探索の履歴に応じて評価関数を適応的に変形

- 目的関数とは異なる評価関数を適応的に変化させつつ局所探索法を繰り返し適用する。
- 直前の局所探索法で得られた局所最適解の構成要素の中でコストが大きな要素にペナルティを加えて評価関数を更新することで、局所最適解から脱出する。
- 目的関数に制約条件に対するペナルティ関数を加えた評価関数を用いて、制約条件のペナルティ重みの更新と局所探索法の適用を交互に適用する**重み付け法**も知られる。



まとめ

- 数理最適化による問題解決と応用事例
- 代表的な最適化問題とアルゴリズム
- 線形計画問題とその解法
- 制約なし非線形計画問題とその解法
- 制約つき非線形計画問題とその解法
- 組合せ最適化問題の難しさ
- 効率的に解ける組合せ最適化問題とその解法
- 整数計画問題に対する厳密解法
- NP困難問題に対する性能保証付き近似解法
- NP困難問題に対する発見的解法

参考文献：数理最適化全般

• 数理最適化の入門書

- ✓ 福島雅夫, 数理計画入門(新版), 朝倉書店, 2011.
- ✓ 久野誉人, 繁野麻衣子, 後藤順哉, 数理最適化, オーム社, 2012.
- ✓ 加藤直樹, 数理計画法, コロナ社, 2008.
- ✓ 山下信雄, 福島雅夫, 数理計画法, コロナ社, 2008.

• 数理最適化問題の定式化

- ✓ H.P.Williams, *Model Building in Mathematical Programming* (5th ed), John Wiley and Sons, 2013.

• 数理最適化の理論の入門書

- ✓ 茨木俊秀, 最適化の数学, 共立出版, 2011.

• ハンドブック

- ✓ 久保幹雄, 田村明久, 松井知己, 応用数理ハンドブック, 朝倉書店, 2002.
- ✓ G.L.Nemhauser, A.H.G.Rinnooy Kan, M.J.Todd (eds), *Optimization*, Elsevier, 1989. (伊理正夫, 今野浩, 刀根薫(監訳), 最適化ハンドブック, 朝倉書店, 1989.)

参考文献：線形計画法，非線形計画法

• 線形計画法

- ✓ V.Chvatal, *Linear Programming*, W.H.Freedman and Company, 1983. (阪田省二郎, 藤野和建, 田口東(訳), 線形計画法(上・下), 啓学出版, 1986, 1988.)
- ✓ 今野浩, 線形計画法, 日科技連, 1987.
- ✓ 並木誠, 線形計画法, 朝倉書店, 2008.

• 非線形計画法

- ✓ 矢部博, 工学基礎 最適化とその応用, 数理工学社, 2006.
- ✓ 田村明久, 村松正和, 最適化法, 共立出版, 2002.
- ✓ 寒野善博, 土谷隆, 最適化と変分法, 2014.
- ✓ 山下信雄, 非線形計画法, 朝倉書店, 2015.
- ✓ 福島雅夫, 非線形最適化の基礎, 朝倉書店, 2001.
- ✓ 金森敬文, 鈴木大慈, 竹内一郎, 佐藤一誠, 機械学習のための連続最適化, 講談社, 2016.
- ✓ S.Boyd, L.Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.
- ✓ J.Nocedal, S.J.Wright, *Numerical Optimization* (2nd ed), Springer, 2006.
- ✓ M.S.Bazaraa, H.D.Sherali, C.M.Shetty, *Nonlinear Programming: Theory and Algorithms* (3rd ed), John Wiley and Sons, 2006.

参考文献：整数計画法と組合せ最適化

• 組合せ最適化全般

- ✓ 久保幹雄, 組合せ最適化とアルゴリズム, 共立出版, 2000.
- ✓ B.Korte, J.Vygen, *Combinatorial Optimization: Theory and Algorithms* (5th ed), Springer, 2012. (浅野孝夫, 浅野泰仁, 小野孝男, 平田富夫 (訳), 組合せ最適化：理論とアルゴリズム (2版), 丸善出版, 2012.)
- ✓ J.Kleinberg, E.Tardos, *Algorithm Design*, Addison-Wesley, 2005. (浅野孝夫, 浅野泰仁, 小野孝男, 平田富夫 (訳), アルゴリズムデザイン, 共立出版, 2008.)
- ✓ C.H.Papadimitriou, K.Steiglitz, *Combinatorial Optimization: Algorithms and Complexity*, Dover, 1982.

• 整数計画問題の定式化

- ✓ 久保幹雄, J.P.ペドロソ, 村松正和, あたらしい数理最適化：Python言語とGurobiで解く, 近代科学社, 2012.

• アルゴリズムの性能と問題の難しさの評価

- ✓ M.R.Garey, D.S.Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W.H.Freeman and Company, 1979.

• ネットワークフロー

- ✓ 繁野麻衣子, ネットワーク最適化とアルゴリズム, 朝倉書店, 2010.
- ✓ R.A.Ahuja, T.L.Magnanti, J.B.Oracle, *Network Flows: Theory, Algorithms, and Applications*, Pearson Education, 2014.

参考文献：整数計画法と組合せ最適化(続き)

• 整数計画問題

- ✓ L.A.Wolsey, *Integer Programming*, Jon Wiley and Sons, 1998.
- ✓ G.L.Nemhauser, *Integer and Combinatorial Optimization*, Jon Wiley and Sons, 1988.

• 性能保証つき近似解法

- ✓ V.V.Vazirani, *Approximation Algorithms*, Springer, 2001. (浅野孝夫 (訳), 近似アルゴリズム, 丸善出版, 2012.)
- ✓ D.P.Williamson, D.B.Shmoys, *The Design of Approximation Algorithms*, Cambridge University Press, 2011. (浅野孝夫 (訳), 近似アルゴリズムデザイン, 共立出版, 2015.)
- ✓ 浅野孝夫, 近似アルゴリズム：離散最適化問題への効果的アプローチ, 共立出版, 2019.

• メタヒューリスティクス

- ✓ 柳浦睦憲, 茨城俊秀, 組合せ最適化 —メタ戦略を中心として—, 朝倉書店, 2001.
- ✓ 久保幹雄, J.P.ペドロソ, メタヒューリスティクスの数理, 共立出版, 2009.
- ✓ E.Aarts, J.K.Lenstra (eds), *Local Search in Combinatorial Optimization*, Princeton University Press, 2003.
- ✓ M.Gendreau, J.-Y.Potvin (eds), *Handbook of Metaheuristics* (3rd ed), Springer, 2019.